

CSCI 499 Senior Project

<MoneyPlanner>

Milleina Ohara

Bachelor of Science in Computer Science

Charleston Southern University

Table of Contents

<MoneyPlanner>

Milleina Ohara

- 1. Statement of Purpose**
- 2. Research Background**
- 3. Hardware / Software Required**
- 4. Final Project Screenshots / Explanation**
- 5. Test Plan**
- 6. Test Results**
- 7. Challenges Overcome**
- 8. Future Enhancements**
- 9. Final Defense Presentation Slides**

1. Statement of Purpose

<MoneyPlanner>

Milleina Ohara

1. Statement of Purpose

Having a plan and organizing one's personal finances is vital to success in life. Yet, organizing your finances is usually not the most exciting thing you do in life. There is a list of things you should do to make sure you stay on top of your bills such as keeping track of expenses as well as creating a budget plan on a monthly basis that is suited just for your life pattern. I personally tend to get disorganized with my finances quite easily and I have used several different financial software applications in the past to help me form good habits. However, none of them really suited for me for a couple of reasons. Firstly, I found that these tools can involve numerous complex operations and take time to learn how to use them properly which leads to a frustration. Secondly, they tended to require you to adjust to spending patterns that are pre-set within the application when it's really your budget that should conform to your life, not the other way around. I also found that the lack of excitement and the inflexibility of financial software interface can discourage users from coming back to the app since organizing finances can be a boring task by itself.

My senior project is financial software application that solves all above issues. MoneyPlanner is a personal finance planner for anyone that wants to keep their finances organized simple and easy. Features like keeping track of expenses and income and creating a budget plan for future expenses can help users manage their finances better. Other special features of this app include customizable categories and saving goals features.

A primary objective of the project though, is to improve the user experience through a both intuitive and interactive user interface (UI) that guides users to helpful information and features as they accomplish tasks. With a clean, exciting, and eye-pleasing interface that puts together some delightful graphs and charts, MoneyPlanner can better demonstrate how you're using your money and gives you more insight into where your money is going. Because every operation within the app can be done intuitively, MoneyPlanner is extremely convenient and easy to use. The intuitive design eliminates the need for users to be trained or refer to documentation.

My project is not only useful for myself, but also beneficial for anybody that wants to stay in control of their finances and be better organize. The project has greatly increased my technical competencies in software design and development in three primary ways. Firstly, I am now much more competent with coding in Java. Java was one of the first programming languages that I have ever dealt with and with its popularity and for my future job, I have developed a desire to become more experienced with it. Second, I have gained experience creating a web application with a servers and database running on the backend from scratch. Having almost no previous experience in dealing with the infrastructure side of computer systems, this was a challenging and intimidating tasks, but it was worth with the challenges and required effort. Third, after taking two of UI/Human-Computer Interaciton (HCI) classes with Dr. Hayes, I've been interested in studying more HCI. I have combined all three of these area for my senior project.

2. Research Background

<MoneyPlanner>

Milleina Ohara

2. Research Background

I have developed MoneyPlanner, which in simple term, is a web-based application that connects to a database server. In order to complete this project, the following knowledge and tools were required:

- Web Servers (Apache HTTP Server and Apache Tomcat) and Database Servers

Grasping the concepts of a two-tier, web client-server architecture with JSP and servlets was one of the most important parts of this project. Proper software and tools were required to establish the desired connection between the Java application and servers. Learning about how servlet mapping works and how it needs to be configured was also crucial. After making sure that stable connection can be made, the concepts of HTTP requests had to be reviewed. Apache HTTP Server mainly handles static contents such as HTML codes whereas Apache Tomcat Server deals with dynamic contents like Java programs. I have learned that combining these two together provides a better structure in terms of performance.

In order to connect my Java application to the MySQL database server, I was required to install MySQL Connector/J software and configure the build path of it within the application. After these steps were done, I needed to learn how to work with series of methods of database interfaces such as connection interface, statement interface, and result set interface in order to retrieve proper dynamic results from database.

- Programming Languages

Java was mainly used as a server-side language in this project in order to perform calculation within the application and also used to extend HTTPServlet in order to retrieve contents from database and forward them to Java Servlet Pages (JSP). HTML and CSS were used to style the user interface. JavaScript was used to dynamically create HTML DOM (Document Object Model) elements depending on the contents of database that was passed.

- Developmental Software

Eclipse IDE for Java EE was used to develop this project. Since I had no experience in using it before, I was required to learn and get used to it.

- Example Features

For my project, I have got the inspiration of the user interface and feature ideas from other popular financial management apps such as mint and ones that are developed in Japan that I used to use personally.

3. Hardware / Software Required for Project

<MoneyPlanner>

Milleina Ohara

3. Hardware / Software Required for Project

Hardware Requirements

See the software requirements below.

Software Requirements

- **User Requirements**

Required Operating System

A 64-bit operating system like Windows 7 or later

Tested environment was on operating system Windows 10

Recommended Web Browsers

Microsoft Internet Explorer

Google Chrome 72

- **Project Server Tools / Requirements**

Web Server

XAMPP Software Package Version 3.2.2

Provides suitable environment for testing MySQL and Apache projects on the local computer.

Includes: Apache HTTP Server Version 2.4

Ideal for plain HTML documents and scripts / programs that are single pages.

Includes: Apache Tomcat v9.0

Gets run as an add-on server to the Apache HTTP server on this project as the Java servlet/JSP container. In this combination, Tomcat executes the Java servlets and JSPs, the Apache serves the static HTML pages.

Database

MySQL Workbench 8.0

- Other Developmental Software / Tools

Java Environment

OpenJDK v1.8.0_191

Java Development Kit for Java Platform which enables developers to develop Java programs and it also includes Java Runtime Environment to run them.

MySQL Connector/J 8.0

JDBC driver (database driver) for MySQL that enables access the MySQL databased from the applications created with Java. The driver can be used with Apache Tomcat.

Eclipse IDE for Java Developers 2018-12

Apache, MySQL and (also PHP) can run on a wide variety of Operating systems such as Linux, Mac OS X, and Microsoft Windows.

Apache module mod_jk 2.4

Apache-Tomcat connector module that lets Apache and Tomcat to work together on the same port 80.

4. Final Project

Screenshots / Explanation

<MoneyPlanner>

Milleina Ohara

User Interface Description

<MoneyPlanner>

Document Version: 1.0

Prepared by: Milleina Ohara
Charleston Southern University

Created: 02/10/2019

Last Updated: 04/10/2019

Table of Contents

1. Introduction	3
2. Project Structure.....	3
3. User Interface Screenshots	

1. Introduction

This document describes the user interface for MoneyPlanner.

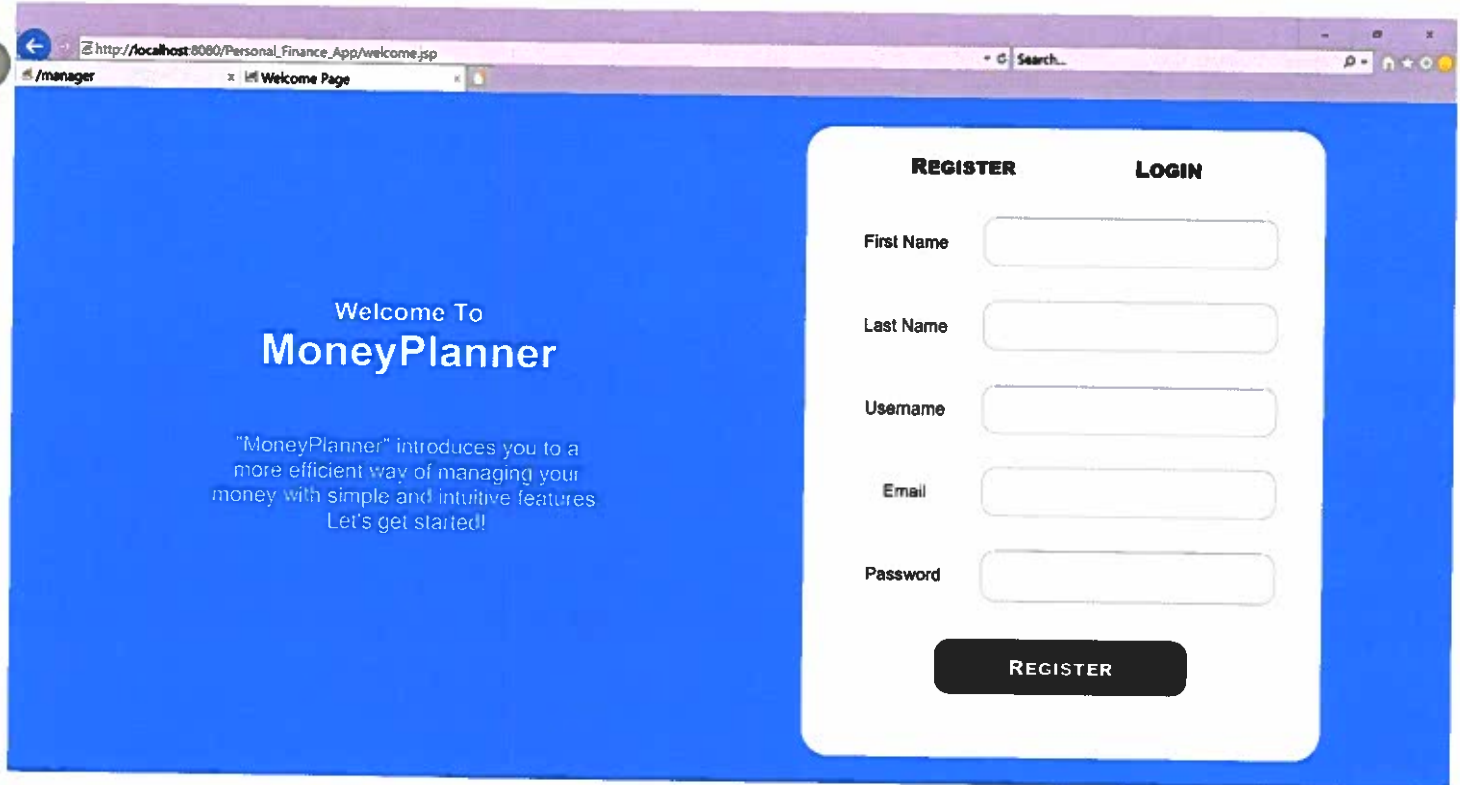
2. Project Structure

Below is the diagram of the structure of MoneyPlanner.

3. User Interface Screenshots / Explanation

Note that the browser used to run MoneyPlanner is Internet Explorer.

1. 2. Register / Login



The screenshot shows a web browser window with the URL `http://localhost:8080/Personal_Finance_App/welcome.jsp`. The page has a blue background. On the left, it says "Welcome To MoneyPlanner" and "MoneyPlanner" introduces you to a more efficient way of managing your money with simple and intuitive features. Let's get started! On the right, there is a white box containing two tabs: "REGISTER" and "LOGIN". Below the tabs are five input fields: "First Name", "Last Name", "Username", "Email", and "Password". At the bottom of the box is a black button labeled "REGISTER".

Document

welcome.jsp

Description

Enables first-time users to create new accounts with MoneyPlanner.

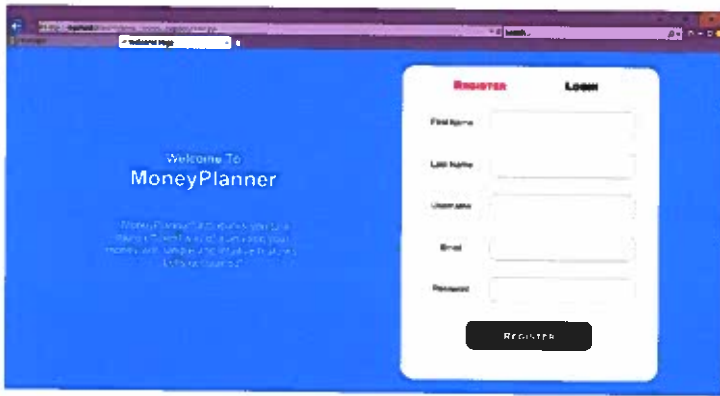
Enables coming-back users to login to their accounts.

Navigation

This is the first page users reach at the start of MoneyPlanner thus there is no navigation needed to reach this page. Navigation to the 'Register Screen' can be done by clicking on the Register tab button on the top left of the input field. Navigation to the 'Login Screen' can be done by clicking on the Login tab button on the top left of the input field.

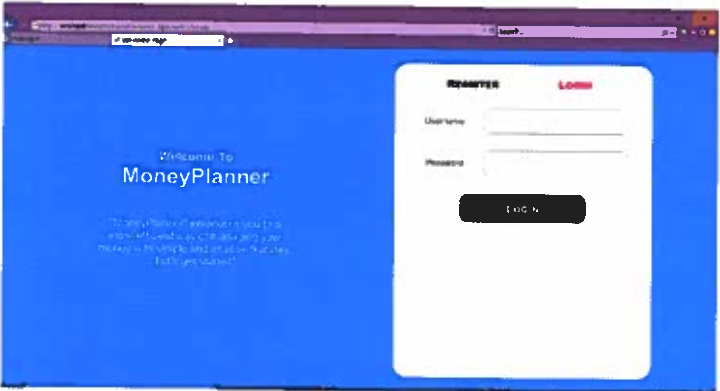
Issues

1. There is no actual email authentication for email inputs. Strings with any length or characters can be accepted and can become serious security issues.
2. There is no 're-type your password' confirmation as of right now.



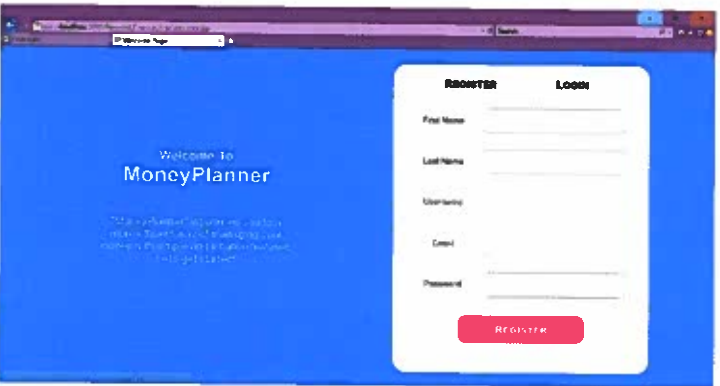
Registration Tab Button Clicked

Registration input fields appear



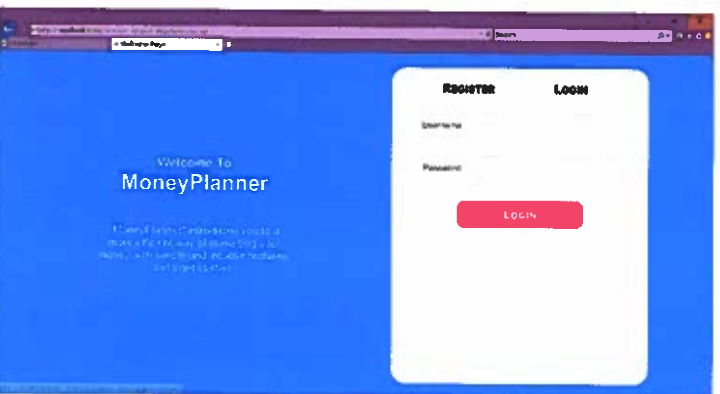
Login Tab Button Clicked

Login input fields appear



Registration Submit Button Hovered

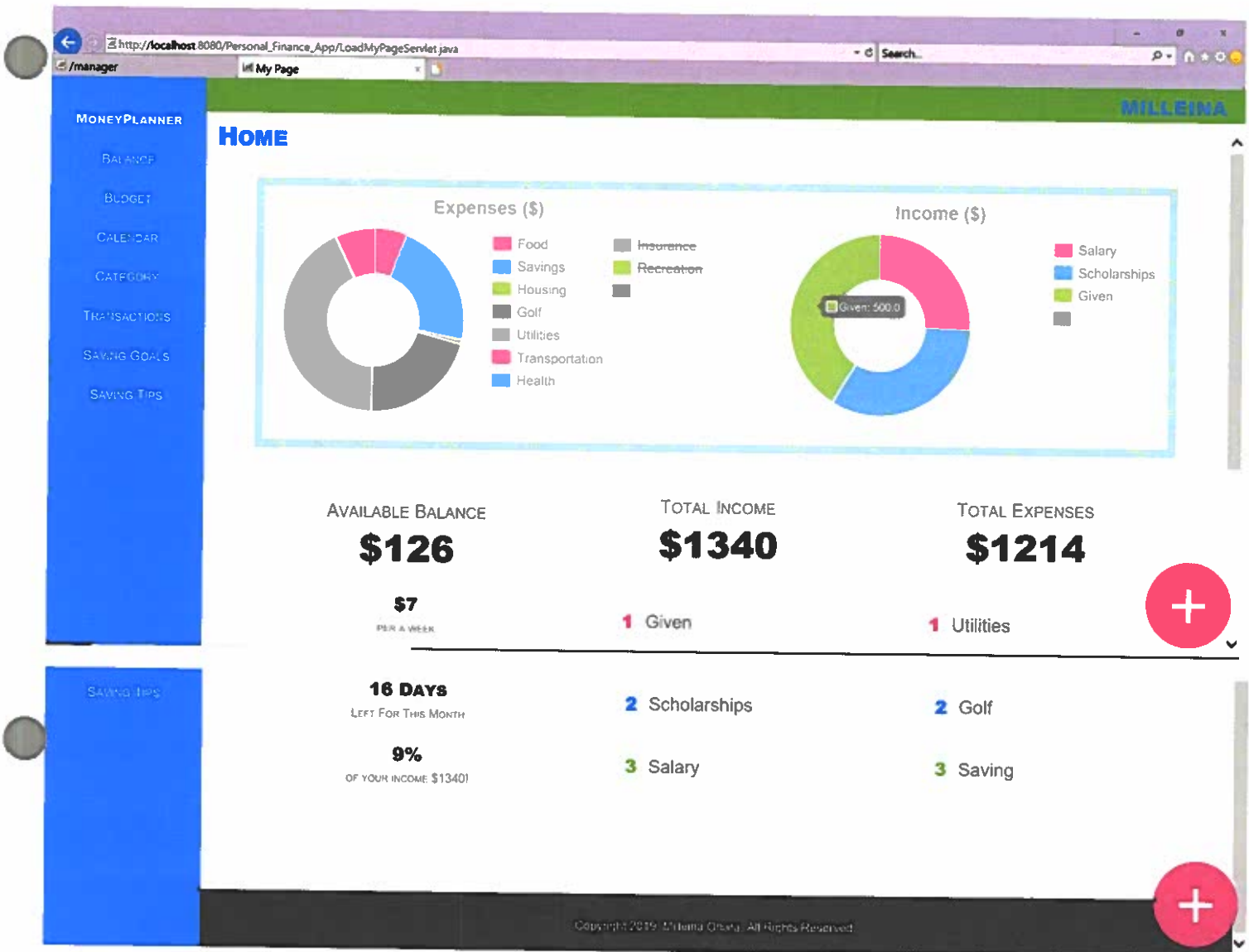
Registration input fields appear



Login Submit Button Hovered

Registration input fields appear

MyPage



Document

my_page.jsp

Description

User's account overview page

Navigation

After the user has been allowed to login to the account and after logged in, by clicking the MoneyPlanner icon on the top left of every page

Issues

1. Ranking List does not change dynamically

User Setting

http://localhost:8080/Personal_Finance_App/UserListServlet.java

Search...

/manager

let User Account Setting

MILLEINA

MONEYPLANNER

BALANCE

BUDGET

CALENDAR

CATEGORY

TRANSACTIONS

SAVING GOALS

SAVING TIPS

USER ACCOUNT SETTING

Logout

First Name Milleina

Last Name Ohara

Username Milleina

Email 12345@gmail.com

Password

EDIT YOUR ACCOUNT INFORMATION

Document

user_info.jsp

Description

User's account setting page

Navigation

By clicking username button in green at the top right of My Page

Issues

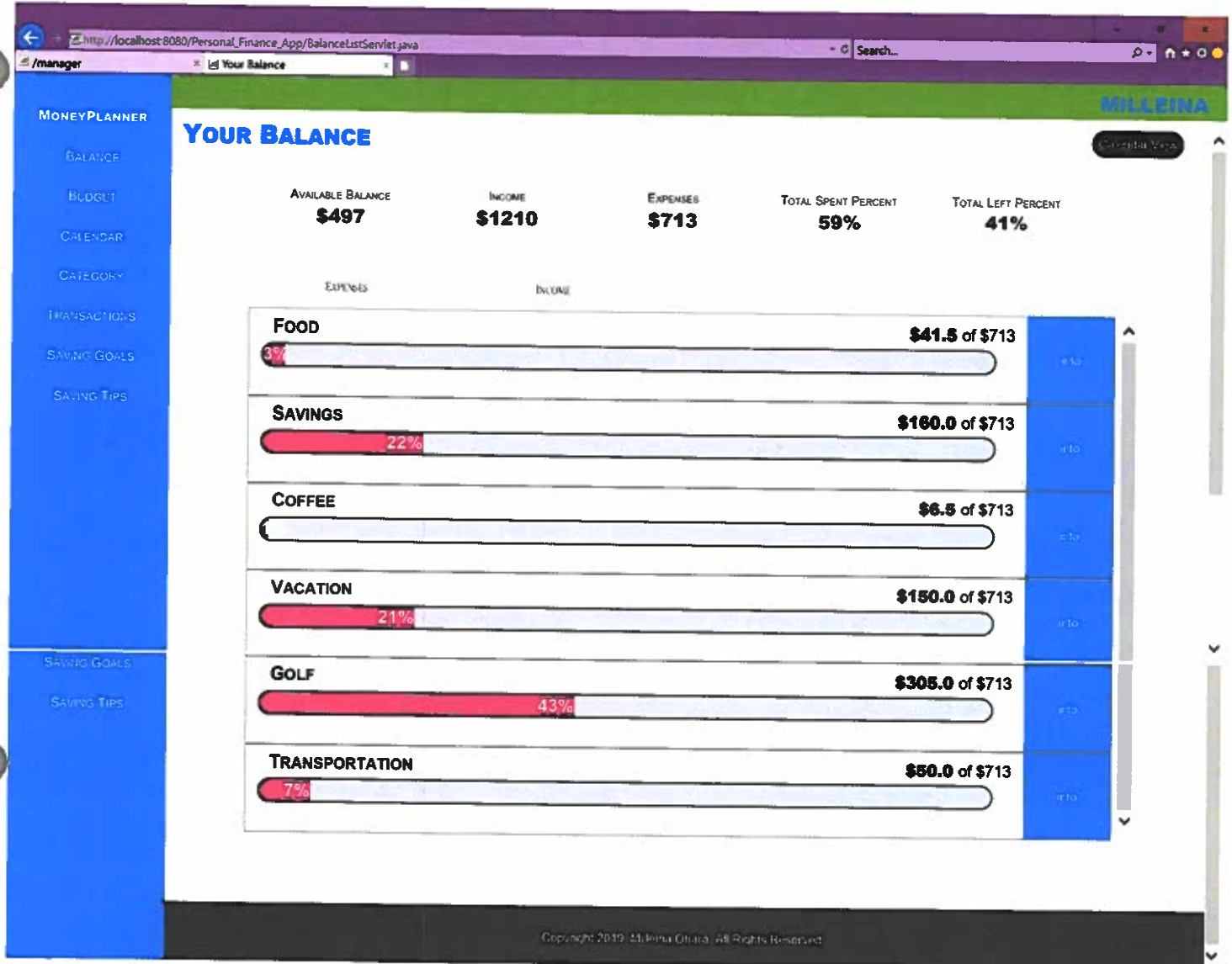
1. Images cannot be uploaded
2. Again, there is no authentication for email or complication level in password so any strings with any characters or length get accepted



Successfully Updated User Account

Displays updated data

Balance



Document

balance.jsp

Description

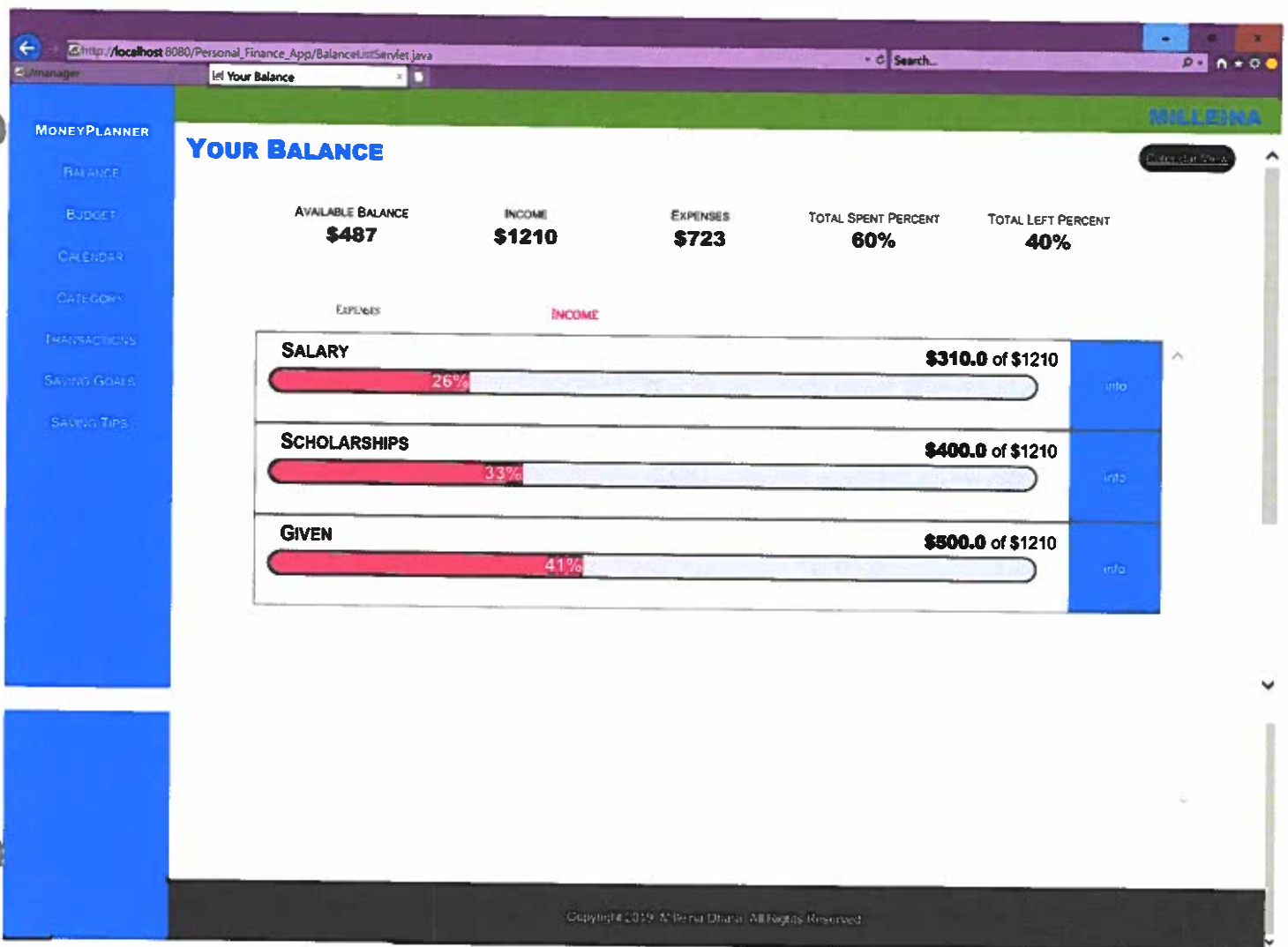
Displays user's balance and expenses in each category

Navigation

By clicking the "Balance" button on the side navigation of MyPage

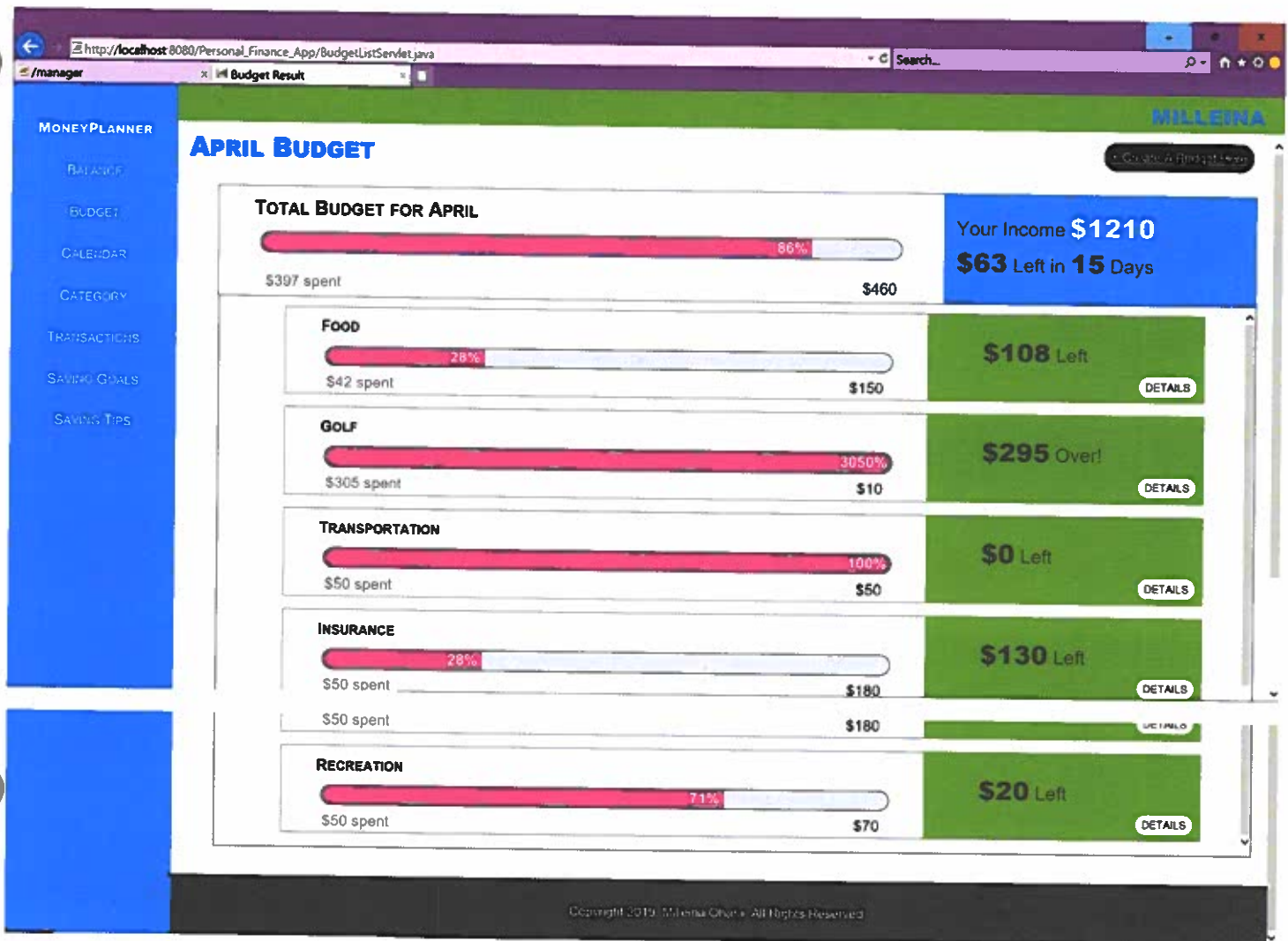
Issues

1. Info buttons are designed to take users to more detailed information about expenses in each category but I did not get to work on it



By clicking on the Expenses/Income tab, users can also view income category's amount as well

Budget



Document

budget.jsp

Description

Displays user's overall budget and budget in each category

Navigation

By clicking the "Budget" button on the side navigation of MyPage

Issues

1. Details button do not work

Add Budget

The screenshot shows a web browser window with the URL `http://localhost:8080/Personal_Finance_App/LoadCategoryAddBudgetServlet.java`. The page title is "Add Budget". On the left is a blue sidebar menu with the following items: MONEYPLANNER, BALANCE, BUDGET, CALENDAR, CATEGORY, TRANSACTIONS, SAVING GOALS, and SAVING TIPS. The main content area has a green header with the text "MILLENA". Below the header, the title "ADD BUDGET" is displayed in blue. The page is divided into two main sections. The left section, titled "Your Category List", contains a grid of category buttons: Food, Savings, Housing, Coffee, Vacation, Golf, Utilities, Transportation, Health, Insurance, Recreation, Cat, Travel, Movies, and another Movies button. The right section, titled "SELECT CATEGORY FROM LEFT", displays the current budget status. It shows "YOUR INCOME FOR APRIL: \$1210" in a dark box. Below this, it says "Current Budget: \$--- (---% of income: \$1210)". The "Updated Budget" is shown as "\$ (---% OF INCOME: \$1210)". There are two "SET" buttons at the bottom right of the main content area. The footer of the page reads "Copyright 2019, Milena Ochoa. All Rights Reserved."

Document

add_budget.jsp

Description

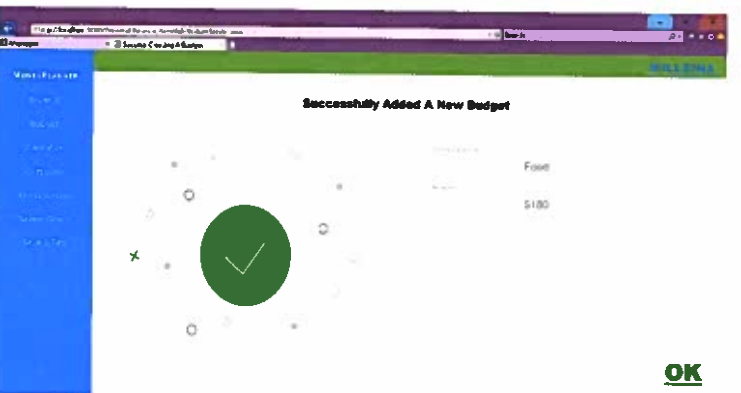
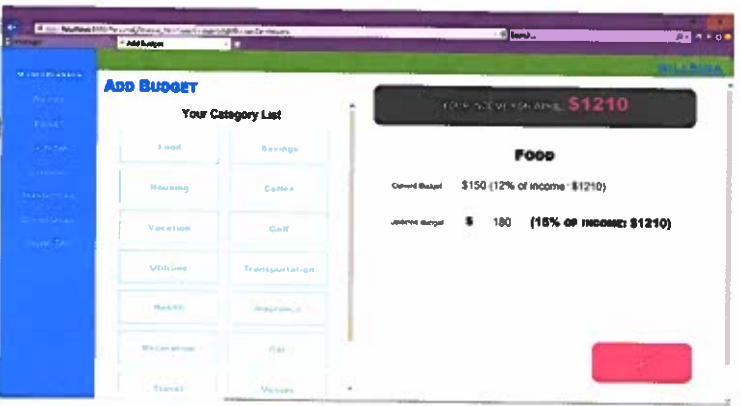
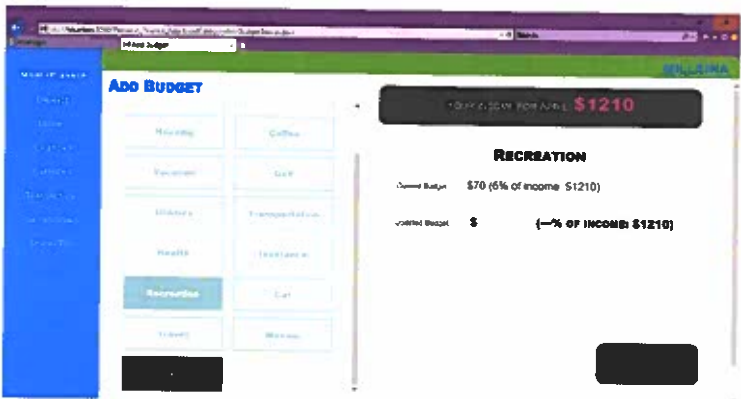
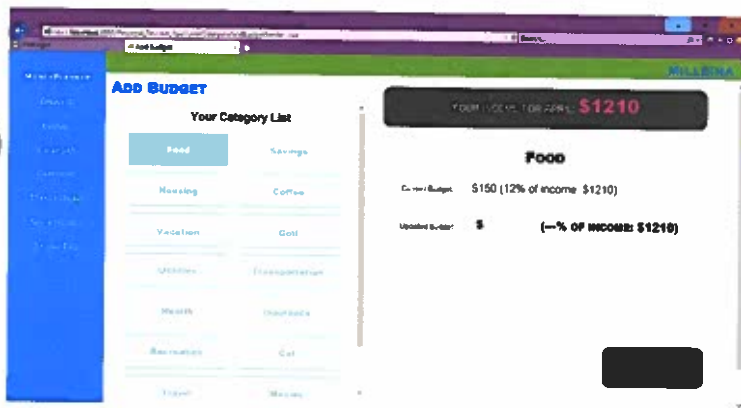
Inserts user's monthly budget of each category into database

Navigation

By clicking the top right "Create A Budget Here" button on the Budget page

Issues

1. No feature to set the overall budget
2. There is no way to set budget of all categories all at once



By clicking on the category button on the left, users can see current budget set for this month for this category

Text Boxes on the right-side show budget amount in dollars and also percentage of income

Users will put in new budget dollar in the text box on the right side and hit "SET" to submit the request

Successfully Created/Updated Budget

Displays updated budget data



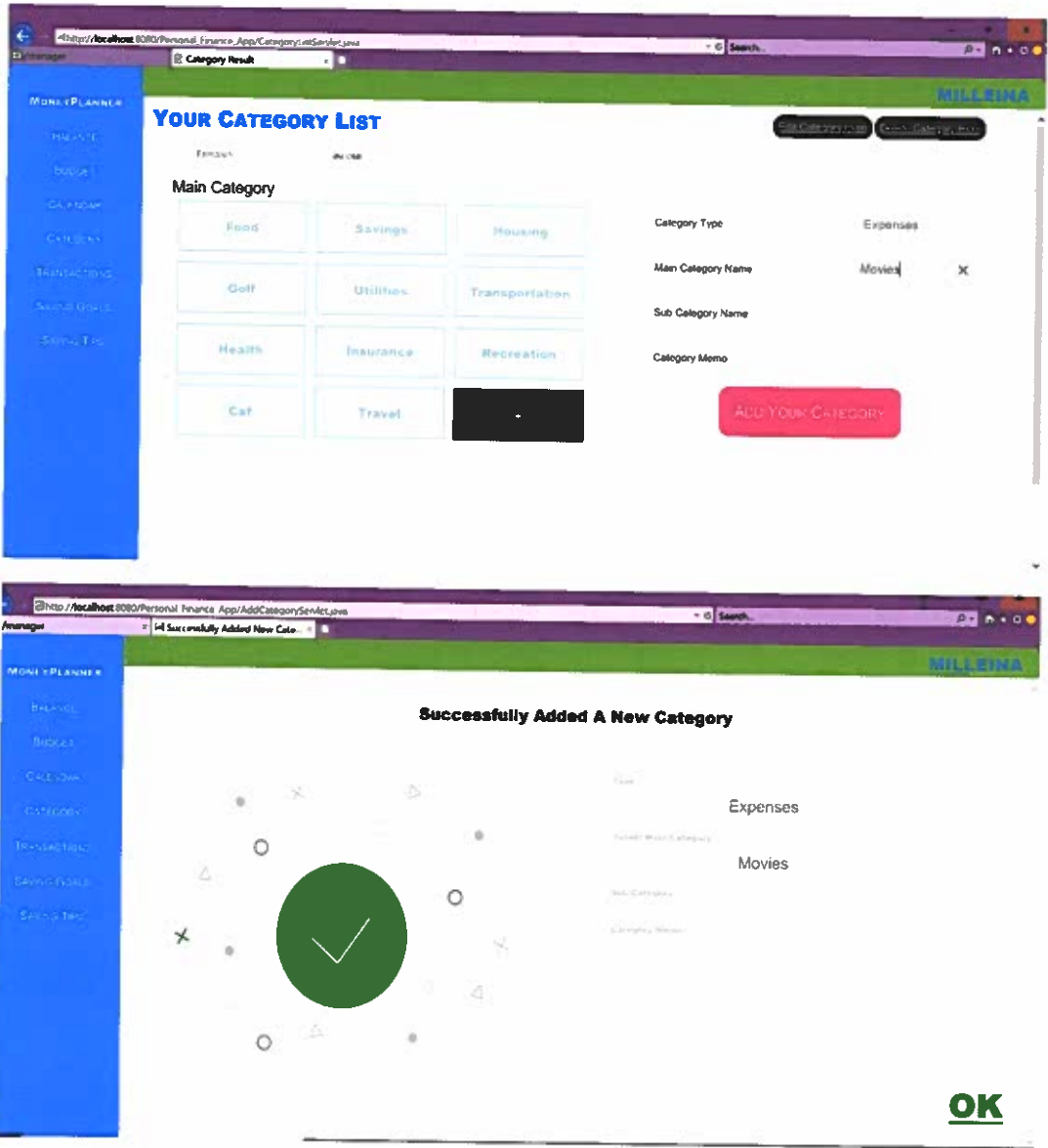
category.jsp

Displays user's category and displays input fields to let them add some more of customized categories

By clicking the “Category” button on the side navigation of MyPage

1. Sub Category name does not get displayed

Add Category



Document

add_category.jsp

Description

Inserts user specific category into database

Navigation

By clicking the + in category list in the Category page

Issues

1. After erroneous inputs and page reload, category does not get retrieved

Edit Category

The screenshot shows a web browser window with the URL `localhost:8080/Personal_Finance/Assets/EditCategoryServlet.action?mainCat=golf`. The page title is "Edit Your Category". On the left is a blue sidebar menu with options: MONEYPLANNER, BUDGET, BUDGET, CATEGORIES, CATEGORY, FINANCIALS, SAVINGS, and SAVING. The main content area has a header "EDIT YOUR CATEGORY" and a sub-header "Choose Category first". Below this is a light blue box labeled "STEP1: Choose a Main Category" containing a dropdown menu with "golf" selected. To the right, there are input fields for "Old Main Category" (golf), "New Main Category Name" (Golf), "New Sub Category Name", and "New Category Description". A red button labeled "EDIT YOUR EXPENSES CATEGORY" is at the bottom right.

The screenshot shows a web browser window with the URL `localhost:8080/Personal_Finance/Assets/EditCategoryServlet.action`. The page title is "Successfully Edited Category". The main content area has a header "Successfully Edited Financial Category" and a large green circle with a white checkmark in the center. To the right, there are input fields for "Old Main Category Name" (golf), "New Main Category Name" (Golf), "New Sub Category Name", and "New Category Description". A green button labeled "OK" is at the bottom right.

Document

`edit_category.jsp`

Description

Updates user specific category into database

Navigation

By clicking the top right "Edit Category Here" button on the Budget page

Issues

1. After erroneous inputs and page reload, category does not get retrieved

Delete Category

DELETE YOUR CATEGORY

Choose Category first

STEP1: Choose a Main Category

Savings

STEP2: Choose a Sub Category

---Choose Sub Category---

DELETE YOUR EXPENSES CATEGORY

DELETE YOUR CATEGORY

Choose Category first

STEP1: Choose a Main Category

Savings

STEP2: Choose a Sub Category

Vacation

DELETE YOUR EXPENSES CATEGORY

Document

delete_category.jsp

Description

Deletes user specific category from database

Navigation

By clicking the top right “Edit Category Here” button on the Budget page

Issues

1. This does not work because of foreign key constraints in database

Transaction

MONEYPLANNER

- BALANCE
- BUDGET
- CALENDAR
- CATEGORY
- TRANSACTIONS
- SAVING GOALS
- SAVING TIPS

TRANSACTIONS

Total Balance: **\$1760** Total Income: **\$4690** Total Expenses: **\$2930** Sort By: **---SELECT---** **GO**

#	DATE	TYPE	DESCRIPTION	CATEGORY	AMOUNT
1	Jan 14, 2019	Expenses	mexican	Food	\$38.0
2	Mar 06, 2019	Expenses	sushi	Food	\$48.0
3	Mar 03, 2019	Expenses	burger	Food	\$74.0
4	Apr 02, 2019	Expenses	mexican	Food	\$12.5
5	Apr 01, 2019	Expenses	hamburgers	Food	\$18.0
6	Mar 11, 2019	Expenses	mexican	Food	\$25.0
7	Apr 09, 2019	Expenses	Pancakes	Food	\$13.0
8	Mar 10, 2019	Expenses	ball	Savings	\$210.0
9	Apr 10, 2019	Expenses	paris	Savings	\$160.0
10	Dec 14, 2018	Income	onecarat	Salary	\$310.0
11	Jan 01, 2018	Income	ippudo	Salary	\$240.0
12	Mar 10, 2019	Income	ippudo	Salary	\$500.0
13	Mar 10, 2019	Income	ippudo	Salary	\$160.0
14	Apr 12, 2019	Income	one carat	Salary	\$130.0
15	Jan 01, 2019	Income	ippudo	Salary	\$100.0
16	Mar 28, 2019	Income	ippudo	Salary	\$136.9
17	Mar 15, 2019	Income	ippudo	Salary	\$500.0
60	Mar 10, 2019	Expenses	pencils	Transportation	\$45.0
61	Apr 01, 2019	Expenses	gas	Transportation	\$25.0
62	Apr 06, 2019	Expenses	gas	Transportation	\$25.0
63	Mar 02, 2019	Expenses	cat food	Insurance	\$35.0
64	Feb 11, 2019	Expenses	bru	Recreation	\$15.0

Copyright 2019, Milene Chana. All Rights Reserved. **GO TO THE TOP**

Document

data_result.jsp

Description

Displays all transactions entered into database

Navigation

By clicking the “Transaction” button on the side navigation of MyPage

Issues

1. Sort By Select Dropdown list does not work

Add Data



Click the + button on the bottom right of MyPage to reach to Enter New Transaction Page

The screenshot shows the 'ENTER NEW TRANSACTION' page. It has a left sidebar with navigation links. The main form area is divided into two steps: 'STEP1: Choose a Main Category' with a dropdown menu showing 'Food', and 'STEP2: Choose a Sub Category' with a dropdown menu showing 'Coffee'. To the right, there are input fields for 'Transaction Type' (set to 'Expenses'), 'Transaction Name' (set to 'starbucks'), 'Date (YYYY-MM-DD)' (set to '2019-04-15'), 'Amount (\$)' (set to '10'), and 'Transaction Memo'. A red button labeled 'Add Your Expenses Transaction' is at the bottom right.

Document

add_data.jsp

Description

Adds new transaction data into database

Navigation

By clicking the bottom right “+” button on MyPage

Issues

1. This does not work because of foreign key constraints in database

Edit Data

EDIT TRANSACTION

Choose Category first

STEP1: Choose a Main Category

Main Category:

Transaction Name To Edit (Optional):

REVEAL EXPENSES DATA TO EDIT

Select the category of which transaction data belongs to

EDIT EXPENSES TRANSACTION

Search By

TYPE: Expenses
MAIN CATEGORY: golf
SUB CATEGORY:
TRANSACTION NAME:

#	DATE	TYPE	DESCRIPTION	CATEGORY	AMOUNT
1	2019-03-11	Expenses	new shirt	golf	\$80.0
2	2019-10-10	Expenses	kiawah round	golf	\$180.0
3	2019-03-01	Expenses	FJ shoes	golf	\$170.0
4	2019-03-16	Expenses	gloves	golf	\$18.0
5	2019-03-09	Expenses	umbrella	golf	\$78.0
6	2019-06-25	Expenses	gloves	golf	\$30.0
7	2019-04-30	Expenses	tee-shirt	golf	\$210.0
8	2019-04-08	Expenses	gloves	golf	\$15.0
9	2019-05-01	Expenses	kiawah	golf	\$130.0
10	2019-04-11	Expenses	plushurst	golf	\$90.0

Current Transaction Data

Date:

New Transaction Data

Date:

GO TO THE TOP

List of transaction in the category gets displayed, select one of them by clicking on the table row

Current Transaction Data

Date: 2019-10-10
Type: Expenses
Transaction Name: kiawah round
Category: golf
Amount (\$): \$180.0

New Transaction Data

Date: 2019-10-15
Type: Expenses
Transaction Name: kiawah round
Category: golf
Amount (\$): \$120.0 x

EDIT YOUR EXPENSES DATA

GO TO THE TOP

Current Transaction Data cannot be edited. Instead, New Transaction Data should be updated according to users' needs

Delete Data

DELETE TRANSACTION

Choose Category first

STEP1: Choose a Main Category

Main Category: Food

Sub Category (Optional): Coffee

Transaction Name To Delete (Optional):

STEP2: Choose a Sub Category

DELETE YOUR CHOICE DATA

Select the category of which transaction data belongs to

DELETE INCOME TRANSACTION

Search By: TYPE: Income, MAIN CATEGORY: Salary, SUB CATEGORY: TRANSACTION NAME: ippudo

#	DATE	TYPE	DESCRIPTION	CATEGORY	AMOUNT
1	2018-01-01	Income	ippudo	Salary	\$340.0
2	2019-03-13	Income	ippudo	Salary	\$500.0
3	2018-03-10	Income	ippudo	Salary	\$180.0
4	2019-01-01	Income	ippudo	Salary	\$100.0
5	2019-03-28	Income	ippudo	Salary	\$130.0
6	2019-04-15	Income	ippudo	Salary	\$180.0
7	2018-05-11	Income	ippudo	Salary	\$400.0

Transaction to Delete

Date: _____

Type: _____

Transaction Name: _____

Category: _____

Amount (\$): _____

DELETE YOUR CHOICE DATA

List of transaction in the category gets displayed, select one of them by clicking on the table row

DELETE INCOME TRANSACTION

Search By: TYPE: Income, MAIN CATEGORY: Salary, SUB CATEGORY: TRANSACTION NAME: ippudo

#	DATE	TYPE	DESCRIPTION	CATEGORY	AMOUNT
1	2018-01-01	Income	ippudo	Salary	\$340.0
2	2019-03-13	Income	ippudo	Salary	\$500.0
3	2019-03-10	Income	ippudo	Salary	\$180.0
4	2019-01-01	Income	ippudo	Salary	\$100.0
5	2019-03-28	Income	ippudo	Salary	\$130.0
6	2019-04-15	Income	ippudo	Salary	\$180.0
7	2018-05-11	Income	ippudo	Salary	\$400.0

Transaction to Delete

Date: 2019-03-28

Type: Income

Transaction Name: ippudo

Category: Salary

Amount (\$): \$130.0

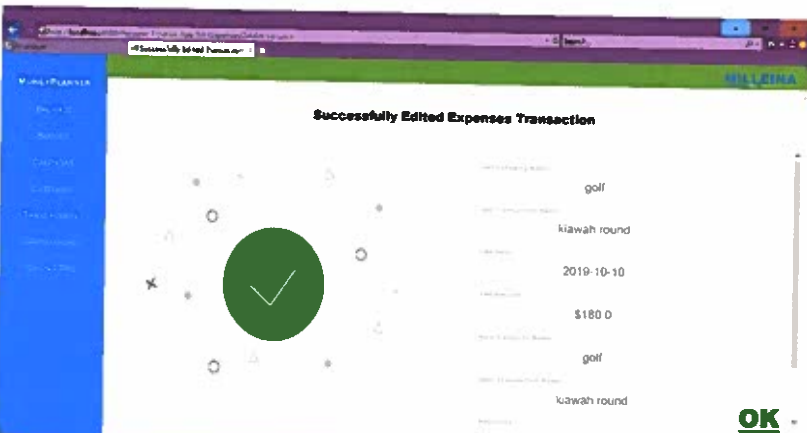
DELETE YOUR CHOICE DATA

Transaction Data cannot be edited. Hit the submit button to confirm deletion of this transaction data

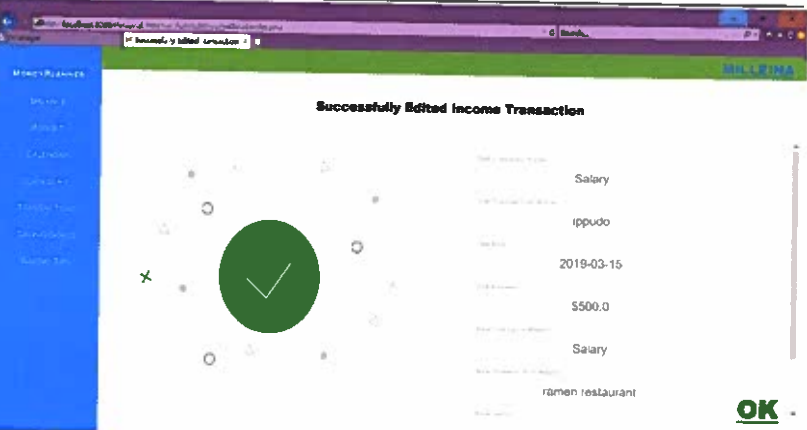
Successfully Added/Edited/Deleted Transaction Data



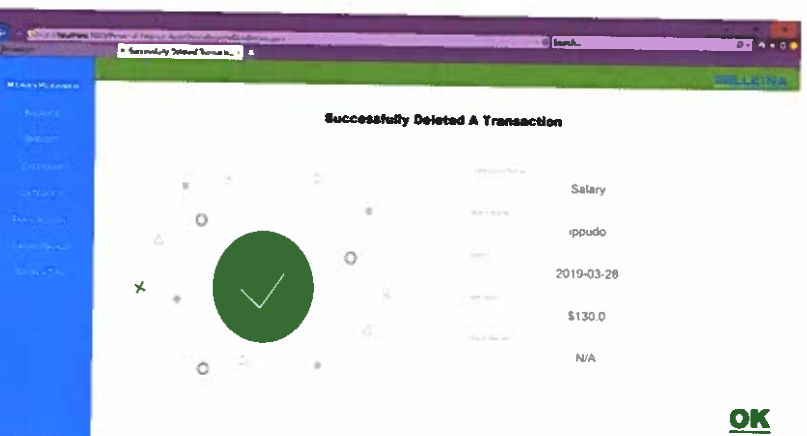
Added New Expenses Transaction



Edited New Expenses Transaction

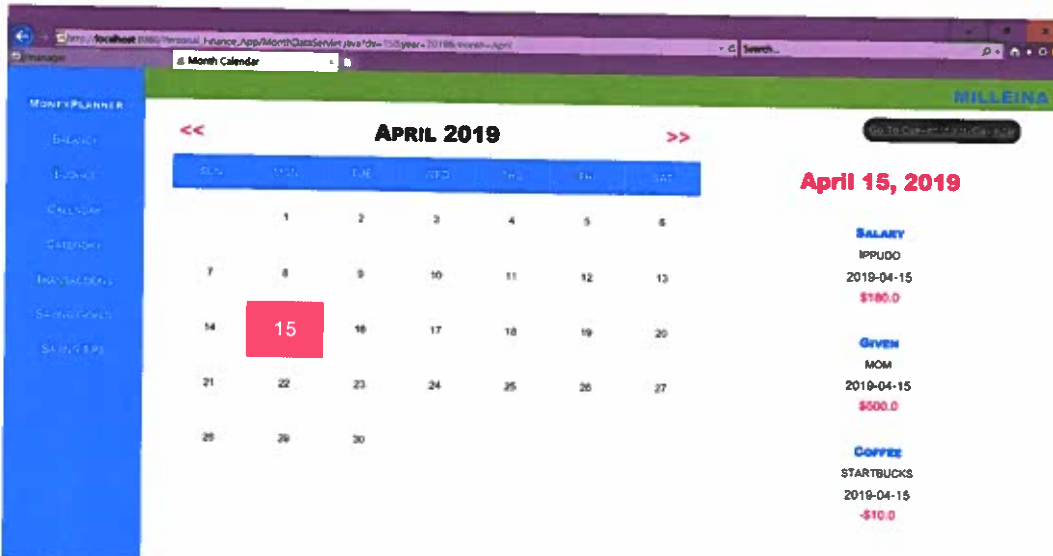
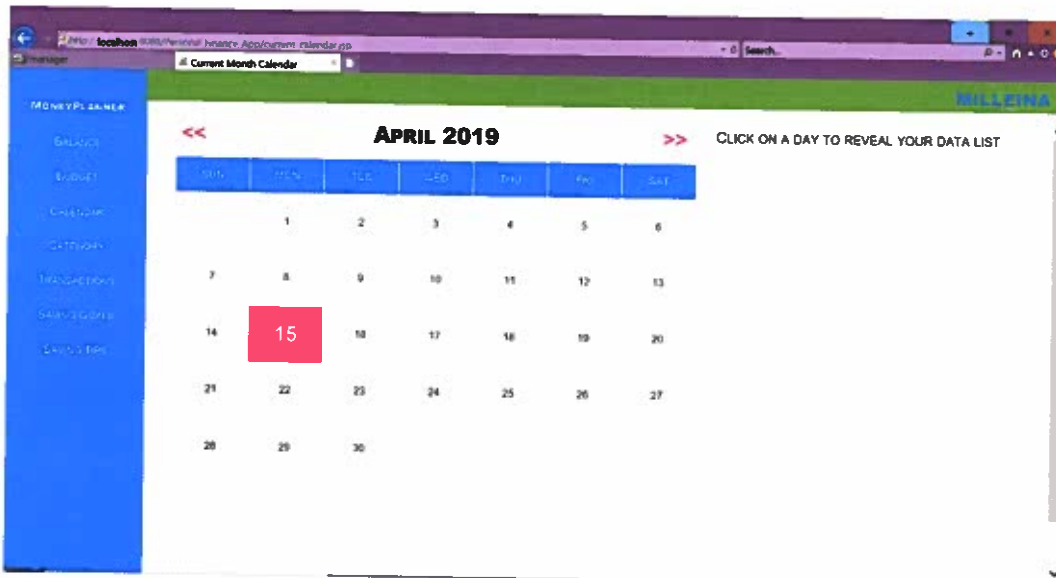


Edited Income Transaction



Deleted Expenses Transaction

Calendar View



Document

current_calendar.jsp, month_calendar.jsp

Description

Transaction data viewed in calendar form

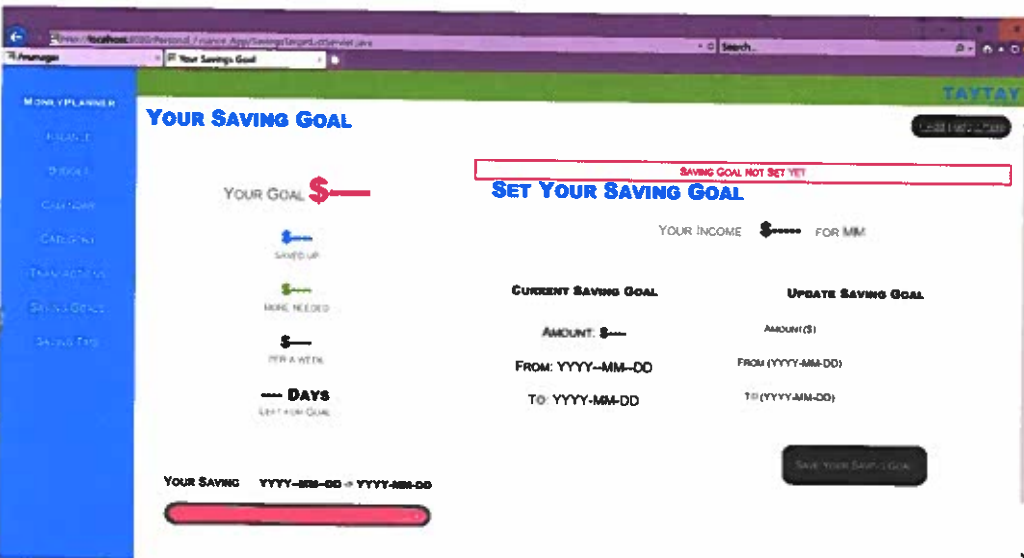
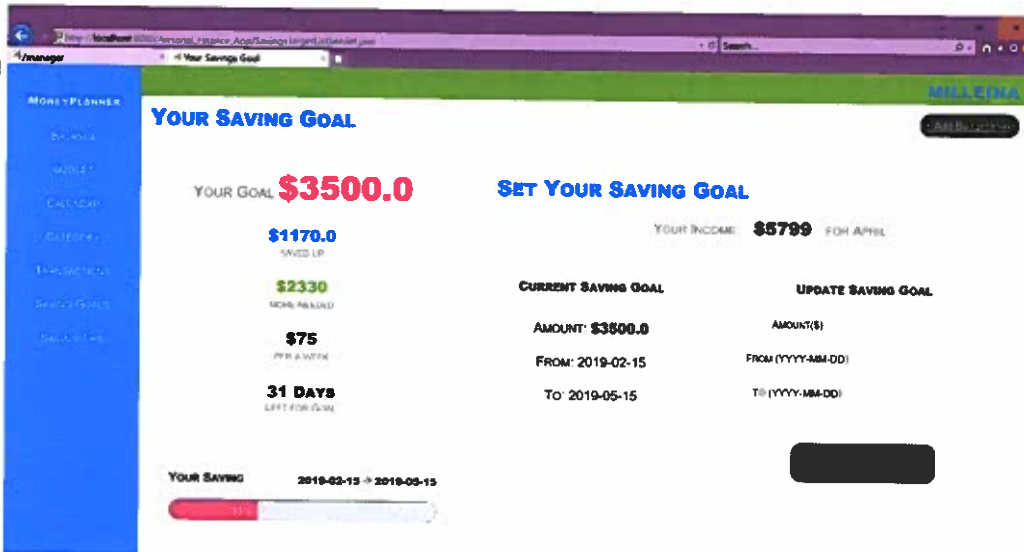
Navigation

By clicking the "Calendar" button on the side navigation of MyPage

Issues

1. No dots to show which day contains transaction and which one does not
2. It will be nice to have a feature where this calendar contains future bills and notifies it to user

Saving Goals



Document

savings_target_home.jsp

Description

Displays user's saving goals if any

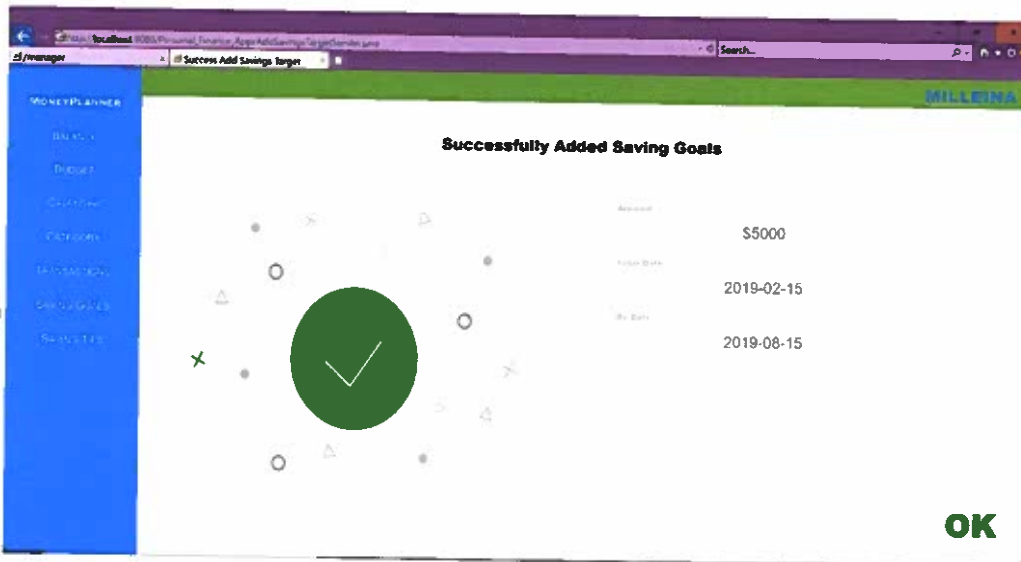
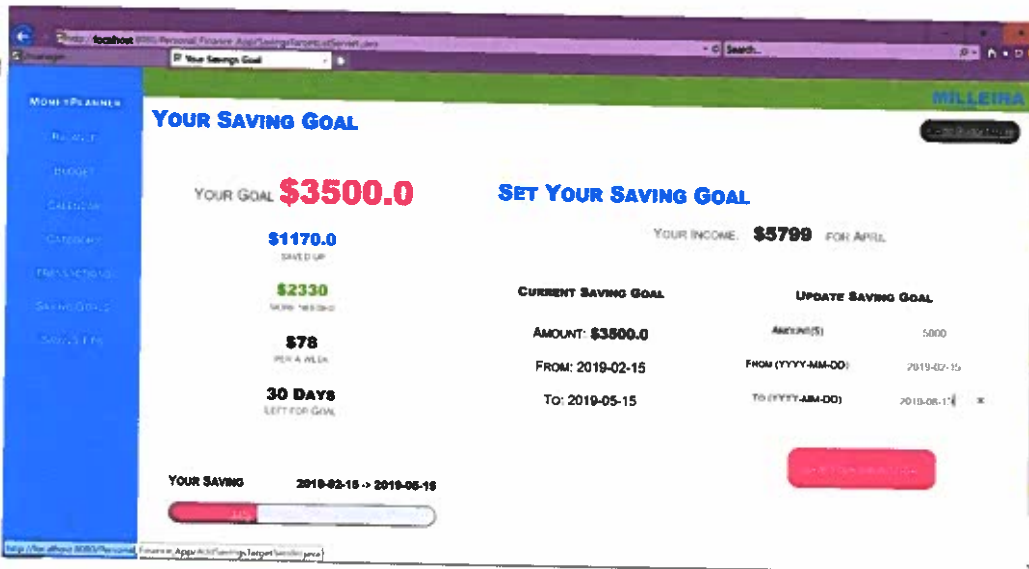
Navigation

By clicking the "Saving Goals" button on the side navigation of MyPage

Issues

1. Line graph for intervals between begin date and end date to show user's progress better

Add Saving Goals



Document

add_savings_target.jsp

Description

Inserts or updates user's saving goals into database

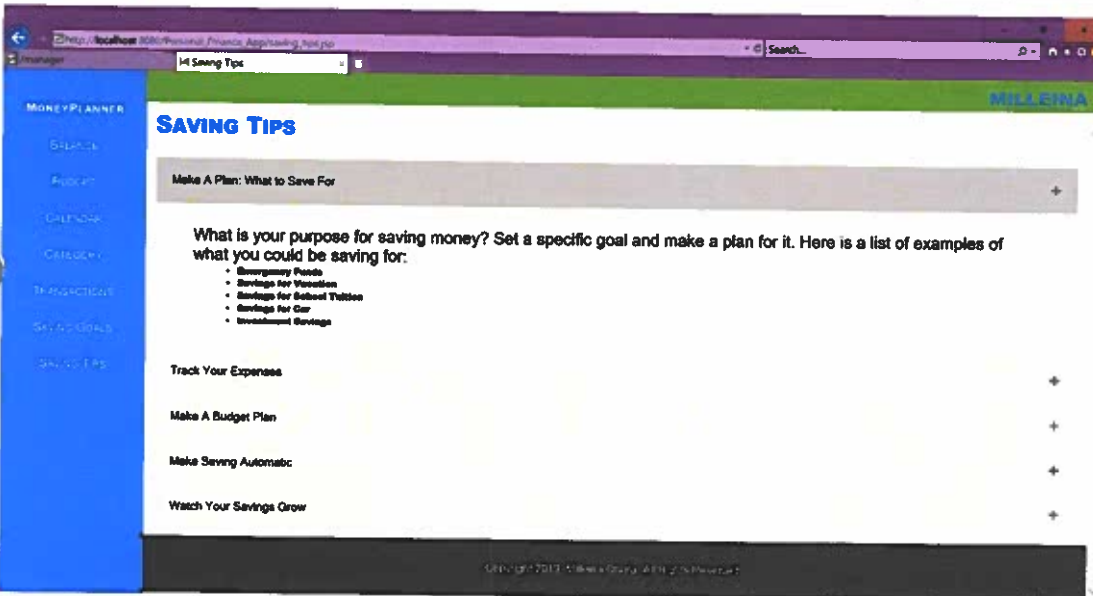
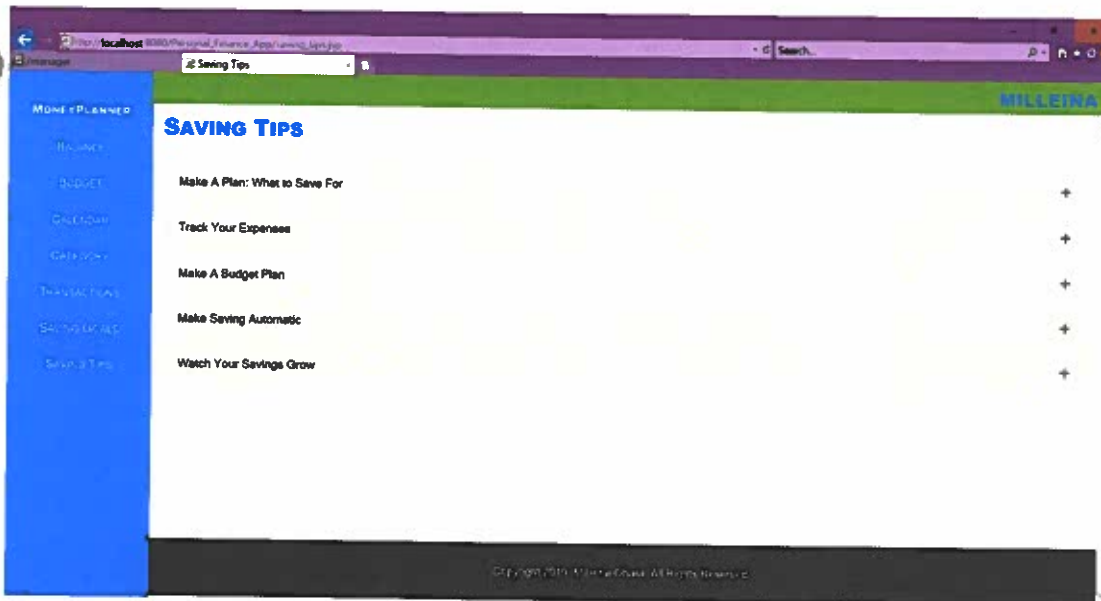
Navigation

Same page as savings_target_home.jsp

Issues

1. Lack of visual during the time period user sets
2. Good to have options like 1month, 3months, 6months intervals for setting goals

Saving Tips



Document

savings_tips.jsp

Description

Displays user’s saving tips in toggle contents

Navigation

By clicking the “Saving Tips” button on the side navigation of MyPage

Issues

5. Test Plan

<MoneyPlanner>

Milleina Ohara

Master Test Plan

<MoneyPlanner>

Document Version: 1.0

Prepared by: Milleina Ohara

Charleston Southern University

Created: 10/10/2018

Last Updated: 03/25/2019

Table of Contents

1. Test Plan Identifier	3
2. References	3
3. Introduction.....	3
3.1. Purpose.....	3
3.2. Objectives.....	3
3.3. Testing Strategy.....	3
3.4. Scope.....	3
4. Features to be Tested.....	4
5. Approach.....	4
5.1. Integration Testing.....	4
5.2. Interface Testing.....	4
5.3. Security Testing.....	4
5.4. Performance Testing.....	4
6. Item Pass/Fail Criteria.....	5
6.1. Data and Database Integrity Testing Criteria.....	5
6.2. Function Testing Criteria.....	5
6.3. User Interface Testing Criteria.....	5
7. Test Environment	5
7.1 Hardware.....	5
7.2 Software.....	5
8. Test Deliverables.....	6
9. Schedule.....	6
10. Responsibilities.....	6
11. Risks.....	6
12. Assumptions and Dependencies.....	6
13. Approvals.....	6
Confidentiality Agreement and Consent Form.....	7
Training Guide {Session Procedure Plan for Session Coordinator.}.....	7
General Overview Debriefing.....	7

1. Test Plan Identifier

MoneyPlanner release 1.0

2. References

List of all documents that support this test plan:

- Project Proposal for MoneyPlanner
- Test Cases for MoneyPlanner
- IEEE 829-1998 Standard for Software and System Test Documentation

3. Introduction

This document describes the procedure and expectations for testing MoneyPlanner Financial Management Web-based application developed by Milleina Ohara.

3.1.Purpose

The purpose of the usability test on MoneyPlanner is to evaluate the usability and functionality of the application. The primary purpose of the application is to help users organize and manage their personal finances more efficiently. The testing procedure asks representative users to operate the application to provide feedback regarding its overall functionality and experience with it. This enables the developer to identify specific problems and improvements to be made on its functionality and user interface which not only reduces the risk of the failure of the product but also increase the users' ability to achieve goals.

3.2. Objectives

The objective of this test plan is to ensure a high level of confidence in the correctness and usefulness of the project deliverables.

3.3. Testing Strategy

The strategy for testing MoneyPlanner is manual tests. Because all the components of this application involve user interaction, they will be tested manually by the developer and representative users.

3.4. Scope

The scope of this Testing activity will include a list of tests stated in the Section 7 below.

4. Features to be Tested

The following is a list of items to be tested within this test plan.

- User Registration and Login
- User Creation and Modification
- Financial Category Creation, Modification, and Deletion
- Financial Data Creation, Modification, and Deletion
- Calendar Interface
- Budget Assignment and Updates
- Savings Target Goals Assignment and Updates
- Saving Tips Interface

5. Approach

The test strategy consists of a series of different tests that will fully exercise the MoneyPlanner financial management system. The primary purpose of these tests is to reveal the systems limitation. A list of the various planned tests and a brief explanation follows below.

5.1. Integration Testing

A detailed set of test cases are to be executed. These test cases will completely and at the very least cover all requirements detailed in the *Test Case for MoneyPlanner*. The tests will be performed manually through the user interface against the fully integrated system containing all components.

5.2. Interface Testing

The user interface on web browser will be tested manually to perform all product tasks and exercise the system requirements; ensuring that all aspects of the display are rendered correctly and that all JSP, HTML, CSS, and JavaScript driven action behave correctly.

5.3. Security Testing

Attempts will be made to access and modify database information through JSP pages without proper authentication to ensure that only authenticated accesses and valid inputs are allowed.

5.4 Performance Testing

Performance testing will be conducted manually. Testers will interact with the user interface to the system and determine if the system responds in a reasonable time. Reasonable is defined as the amount of time a data entry person would expect the system to respond in, which is if the tester can measure the time, in minutes or hours, with a wall clock the system fails performance testing.

6. Item Pass/Fail Criteria

- If an item tested for all four of available participants in this study and do not work for at least one participant, then it will be considered as fail case
- System crash will be considered as fail case at all time
- After submitting query in the system, if expected page would not appear then it will be considered as fail case
- To an item test to pass the above criteria should be working properly

6.1.Data and Database Integrity Testing Criteria

All database access methods and processes function as designed and without any data corruption.

6.2. Function Testing Criteria

All planned tests have been executed. All identified defects have been addressed.

6.3.User Interface Testing Criteria

Each window successfully verified to remain consistence within acceptable standard.

7. Test Environment

7.1. Hardware

There are no specific hardware requirements for this *Test Plan*.

7.2. Software

The following software elements are required in the test environment for this *Test Plan*.

Software Element Name	Type and Other Notes
A 64-bit operating system like Windows 7 or later	Operating System
Google Chrome 72	Internet Browser
Xampp v3.2.2	Cross-platform web server package
Apache HTTP Server v2.4	HTTP Web Server
Apache Tomcat v9.0.14	Application Server
MySQL v8.0	Database
OpenJDK v1.8.0_191	Java Development Kit for Java Platform

8. Test Deliverables

List of all test deliverables:

- Test Plan for MoneyPlanner
- Test Case for MoneyPlanner
- Defect Tracking Log for MoneyPlanner

9. Schedule

All listed Tests in Section 7

3/30-4/1

10. Responsibilities

All roles and processes including designing, preparing, executing, checking, and resolving test activities in MoneyPlanner project are done by Milleina Ohara.

11. Risks

Risk	Mitigation Strategy
Inability to complete tasks at the estimated time	Track the progress of a project on daily basis
Unavailable prerequisites	Make sure to meet prerequisites
Unexpected expansion of scope	Include contingencies in schedule
Test data proves to be insufficient.	• Redefine test data • Review Test Plan and modify components

12. Assumptions and Dependencies

Test plan will have the following dependencies:

- Tester and representative participants will need to be present during the schedule for executing the test cases
- The source code must be unit tested and provided within by the start date given in the schedule for executing the test cases
- The test environment described in Section 10 will need to be available by the start date given in the schedule for executing the test cases

13. Approvals

<i>Name</i>	<i>Signature</i>	<i>Date</i>

Confidentiality Agreement and Consent Form

Use Charleston Southern University IRB format

- Informed consent guidelines
- Informed consent form

Training Guide *{Session procedure plan for session coordinator.}*

The Personal Finance Planner Testing Background Information. (10min.)

General Description: This section will be used to orient the participants to MoneyPlanner.

1. Present the personal computer with MoneyPlanner ready to be used on.
2. Demonstrate how to register and login to the personal account.
3. Demonstrate how to add, edit and delete financial data within the application.
4. Demonstrate how to view data day by day through calendar view.
5. Demonstrate how to add a new category, edit and delete existing categories within the application.
6. Demonstrate how to initialize and update the budget within the application.
7. Demonstrate how to initialize and update the saving goals within the application.
8. Present the saving tips page of the application.

General Overview Debriefing:

{Remember to write the participant's responses to your clarification or debriefing questions down.}

1. Ensure that participant has answered all questions on the questionnaire.
2. Ensure that you can read any written responses. If the writing is questionable, please re-write.
3. Review the individual questions with the participant.
 - Ensure that you understand the operator's actual level of electronical device usage.

Consent to Participate in Research

Title of Research:

Usability Test for the Personal Financial Planner

What is the purpose of the research?

The purpose of the usability test on Personal Financial Planner is to evaluate the usability and functionality of the application. The testing procedure asks representative users to operate the application to provide feedback regarding its overall functionality and experience with it. This experiment enables the developer to identify specific problems and improvements to be made on its functionality and interface to increase the users' ability to achieve goals.

For how long am I expected to be in this study and how much of my time is required?

30 minutes approximately. After the experiment, you are asked to fill out a questionnaire regarding the usability of the application.

When and where is this study to take place?

Test procedure takes place anywhere that is moderately quiet where there will be minimum distractions for the operator. The study takes place when it is convenient for each representative user.

How many participants will be in this study?

There will be total 4 CSU students participating in this study.

What are the anticipated benefits of participating in this study?

You are introduced to a useful and interactive finance management application tool that helps you greatly. With this tool, you will be able to have your finances organized at all time.

Will I be compensated for participating in this study?

No. There will be no compensation for this study.

How will my confidentiality be protected?

The voice recorded will only be used for data collecting purpose only and you will not be identified by the comments that you will make on the application.

Is my participation voluntary?

Yes. Your participation is voluntary.

Can I stop participating any time during this research without penalty?

Yes. As long as you return the electronic device provided, you are free to stop participating in this study any time.

How can I withdraw from this study if I choose not to participate any longer?

You will be asked to inform the tester that you wish to withdraw from the study and return the provided device before you withdraw from the study.

Will I receive a copy of this informed consent document?

Yes, you are going to receive the copy of this document.

Who should I contact if I have questions regarding this study?

You should contact the investigator of this study.

Have you read and fully understood the informed consent above? **Yes** **No**

Your name and signature below indicates you have read the information above, you have freely decided to participate in this research, and you certify that you are at least 18 years old.

Participant Name (please print):

Participant Signature:

Date:

Investigator Signature:

Date:

Personal Finance Planner Evaluation Questionnaire

1. How satisfied are you with the reliability of this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
2. How satisfied are you with the consistency of this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
3. How satisfied are you with the security of this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
4. How satisfied are you with the software's ease of use?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
5. How satisfied are you with the ability of the software to complete the task effectively?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
6. How satisfied are you with the look and feel of this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
7. How satisfied are you with the ease of learning process to use this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied

8. How satisfied are you with the account setup experience of this software?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
9. How satisfied are you with the clearness of error messages of this software after invalid inputs or erroneous operations?
 - a. Extremely satisfied
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all satisfied
10. How likely is that you would recommend this software to a friend or family member?
 - a. Extremely likely
 - b. Very satisfied
 - c. Somewhat satisfied
 - d. Not so satisfied
 - e. Not at all likely
11. Do you have any thoughts on how to improve this software?

6. Test Results

<MoneyPlanner>

Milleina Ohara

Test Case for MoneyPlanner

Milleina Ohara

Charleston Southern University

1. Introduction

2. Test Cases

2.1. <Registration / User Setting / Sign Out / Sign In >

- 2.1.1. <Registration>
- 2.1.2. <User Setting>
- 2.1.3. <Sign Out>
- 2.1.4. <Sign In>

2.2. <Add / Edit / Delete Financial Data>

- 2.2.1. <Add Data>
- 2.2.2. <Edit Data>
- 2.2.3. <Delete Data>

2.3. <Calendar View>

- 2.3.1. <Calendar View>

2.4. <Financial Category>

- 2.4.1. <Category List>
- 2.4.2. <Add Category>
- 2.4.3. <Edit Category>
- 2.4.4. <Delete Category>

2.5. <Initialize / Update Budget>

- 2.5.1. <Initialize/Update Budget>

2.6. <Initialize / Update Saving Goals Target>

- 2.6.1. <Initialize/Update Budget>

2.7. <Saving Tips>

- 2.7.1. <View Saving Tips>

1. Introduction

The purpose of this Test Case document is to specify and communicate the specific conditions which need to be validated to enable an assessment of MoneyPlanner.

****Note** that these tests are conducted on web browser, Internet Explorer.

2. Test Cases

2.1. <Sign Up / User Setting / Sign Out / Sign In >

2.1.1. <Sign Up>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_001	Sign Up	High	Test the Registration Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	Apache Tomcat Server running on localhost port 8080
2	Access to Chrome Browser

Test Scenario

Verify on entering valid first name, last name, username, email, and password, the user can sign up for the MoneyPlanner

S#	Variable	Test Data	Notes
1	First Name Last Name Username Email Password	<Charleston Southern> <Univ.> <CSU> <csu@csuniv.com> <loveCHS>	There should be no invalid inputs for first name, last name, email and password.
2	First Name Last Name Username Email Password	<Charleston Southern> <Univ.> <milleina> <csu@csuniv.com> <loveCHS>	If username is already taken, it is considered an invalid username

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to the registration page	Site should open		
2	Enter First Name, Last Name, Username, Email Address, and Password	Credentials can be entered		
3	Click "Sign Up" Button	- User now has his or her own account for MoneyPlanner, and is logged in - Should see the main page of site - If any of required input field has been left blank, then show error message to enter values		

2.1.2. <User Setting>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_002	User Setting	Medium	Test the User Setting Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	Apache Tomcat Server running on localhost port 8080
2	Access to Chrome Browser
3	User has a registered account
4	User is signed in

Test Scenario

Verify on entering valid user information, the user can update his or her user information within the MoneyPlanner

S#	Variable	Test Data	Notes
1	First Name Last Name Username Email Password	<Computer> <Science> <CSUniversity> <csu@csuniv.com> <loveCharleston>	There should be no invalid inputs for first name, last name, email and password.
2	First Name Last Name Username Email Password	<Charleston> <Southern> <milleina> <charleston@csuniv.com> <loveCHS>	If username is already taken, it is considered an invalid username

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	From the main page, navigate to user setting page by clicking user's name on the top right of the site	User Setting page should open		
2	Enter Test Data	Credentials can be entered		
3	Click "Update User Information" Button	- User's information should be updated within MoneyPlanner - One of the two messages should be shown: "Successfully Updated" or "Error"		
4	Click "Ok"	Show the updated user information		

2.1.3. <Sign Out>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_003	Sign Out	High	Test the Sign Out Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	Apache Tomcat Server running on localhost port 8080
2	Access to Chrome Browser
3	User has a registered account
4	User is signed in

Test Scenario

Verify on pressing “Log Out” Button, the user can sign out from the MoneyPlanner

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	If not already there, navigate to the user setting page following the steps from test case 2.1.2.	User Setting Page should open		
2	Click “Log Out” Button	- User is now logged out - Redirect to the sign up / sign in page		

2.1.4. <Sign In>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_004	Sign In	High	Test the Log In Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	Apache Tomcat Server running on localhost port 8080
2	Access to Chrome Browser
3	User has a registered account
4	User is signed out

Test Scenario

Verify on entering valid username and password, the user can sign in for the MoneyPlanner

S#	Variable	Test Data	Notes
1	First Name Last Name Username Email Password	<Computer> <Science> <CSUniversity> <csu@csuniv.com> <loveCharleston>	There should be no invalid inputs for first name, last name, email and password.
2	First Name Last Name Username Email Password	<Computer> <Science> <CSU> <csu@csuniv.com> <loveCharleston>	If username is already taken, it is considered an invalid username

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to sign in page	Site should open		
2	Enter Username and Password	Credentials can be entered		
3	Click "Sign In" Button	- User is now logged in - Should see the main page of site - If any of required input field has been left blank, then show error message to enter values		

2.2. <Add / Edit / Delete Financial Data>

2.2.1. <Add Data>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_005	Add Data	High	Test the “Add A New Financial Data” Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page

Test Scenario

Verify on entering valid financial data information, the user can add a new financial data to the MoneyPlanner (Data both in default categories and user-specific categories)

Notes

- All fields except for “Data Memo” are required fields
- Invalid inputs for fields Main or Sub category would be selecting “--Choose Main/Sub Category--” as category names
- Invalid inputs for field “Date” would be non-string or any string that doesn’t follow “YYYY-MM-DD” format
- Invalid inputs for field “Amount” would be anything non-numerical and non-positive.

S#	Variable	Test Data
1	Main Category Data Name Date Amount Data Memo	<Food> <Mexican> <2019-03-15> <\$8.5> N/A
2	Main Category Sub Category Data Name Date Amount Data Memo	< Savings> <Vacation> <Hawaii> <2018-02-01> <\$200> N/A
3	Main Category Sub Category Data Name Date Amount Data Memo	<Golf> <gloves> <2019-08-15> <\$15> N/A
4	Main Category Sub Category Data Name Date Amount Data Memo	<Food> <Asian> <Chinese> <2019-05-15> <\$8.5> N/A
5	Main Category Sub Category Data Name Date Amount Data Memo	<Salary> <Ippudo> <2019-06-15> <\$400> N/A

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to add data page by clicking the + button on the bottom right of the main page	Site should open		
2	Choose the type of the new financial data by clicking either the Expenses / Income tab on the top	- Clicking the Expenses Tab should display input fields for adding new expenses financial data - Clicking the Income Tab should display input fields for adding new income financial data		
3	Choose Main Category from the dropdown lists located on the left-hand side of the page	Clicked variable in dropdown should be selected as main category where new financial data is going to belong, and now waiting to be passed as input parameters for Servlet (No changes on user view)		
4	(Choose Sub Category from the dropdown lists located on the left-hand side of the page, only if necessary)	Clicked variable in dropdown should be selected as sub category where new financial data is going to belong, and now waiting to be passed as input parameters for Servlet (No changes on user view)		
5	Enter Data Name, Date, Amount, and Data Description	Credentials can be entered		
6	Click "Add Your Expenses / Income Data" Button	One of the two messages should be shown: "Successfully Added" or "Error"		
7	Click "Ok"	- Redirect to main page with the added financial data reflected on charts		

2.2.2. <Edit Data>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_006	Edit Data	High	Test the “Edit Existing Financial Data” Functionality in MoneyPlanner	Pass... and Fail

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	Financial data entered by this user exists

Test Scenario

Verify on entering valid financial data information, the user can edit existing financial data in the MoneyPlanner,
(Data both in default categories and user-specific categories)

Notes

- Input constraints are the same as ones specified in “Note” section of 2.2.1
- Choosing a valid main category variable from dropdown list is required
- Choosing sub category from additional dropdown list (if there is any) and data name are not required. However, with valid inputs of these fields will narrow down the list of data to edit

S#	Variable	Test Data (Old)	Test Data (New)
1	Main Category Data Name Date Amount Data Memo	<Food> <Mexican> <2019-03-15> <\$8.5> N/A	<Food> <Hamburgers> <2019-03-15> <\$8.5> N/A
2	Main Category Sub Category Data Name Date Amount Data Memo	<Savings> <Vacation> <Hawaii> <2018-02-01> <\$200> N/A	<Savings> <Hawaii> <2018-02-01> <\$200> N/A
3	Main Category Sub Category Data Name Date Amount Data Memo	<Golf> <gloves> <2019-08-15> <\$15> N/A	<Golf> <gloves> <2019-08-15> <\$-10> N/A
4	Main Category Sub Category Data Name Date Amount Data Memo	<Food> <Chinese> <2019-05-15> <\$8.5> N/A	<Food> <Asian> <Chinese> <2019-05-15> <\$8.5> N/A
5	Main Category Sub Category Data Name Date Amount Data Memo	<Salary> <Ippudo> <2019-06-15> <\$400> N/A	<Salary> <Ippudo> <2019-06-15> <\$700> N/A

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Firstly, navigate to add data page by clicking the + button on the bottom right of the main page	Add Data page should open		
2	Navigate to edit data page by clicking the "Edit Data" button on the top right	Edit Data page should open		
3	Choose the type of the new financial data by clicking either the Expenses / Income tab on the top	- Clicking the Expenses Tab should display input fields for editing expenses financial data - Clicking the Income Tab should display input fields for editing income financial data		
4	Choose Main Category from the dropdown lists located on the left-hand side of the page	Clicked variable in dropdown should be selected as main category and show in the user input field labeled as main category		
5	(Choose Sub Category from the dropdown lists located on the left-hand side of the page, only If necessary)	Clicked variable in dropdown should be selected as sub category and show in the user input field labeled as sub category		
6	(Type Data Name to edit if the user already knows which financial data he or she wishes to edit)	Credentials can be entered		
7	Click "Reveal List of Expenses / Income Data to Edit" Button	After a page reload, list of expenses / income that can be edited shows up - If sub category input field is left blank and gets submitted, data list should show every data that is the selected main category's sub category as well Any lists should be clickable		
8	Click one of the lists that needs to be edited			
9	Click "Edit Your Expenses / Income Data" Button	One of the two messages should be shown: "Successfully Updated" or "Error"		
10	Click "Ok"	Redirect to main page with the updated financial data reflected on charts		

2.2.3. <Delete Data>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_007	Delete Data	High	Test the “Delete Existing Financial Data” Functionality in MoneyPlanner	Pass... and Fail

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	Financial data entered by this user exists

Test Scenario

Verify on entering valid financial data information, the user can delete existing financial data in the MoneyPlanner,
(Data both in default categories and user-specific categories)

Notes

- Input constraints are the same as ones specified in “Note” section of 2.2.1
- Choosing a valid main category variable from dropdown list is required
- Choosing sub category from additional dropdown list (if there is any) and data name are not required. However, with valid inputs of these fields will narrow down the list of data to delete

S#	Variable	Test Data
1	Main Category Data Name Date Amount Data Memo	<Food> <Mexican> <2019-03-15> <\$8.5> N/A
2	Main Category Sub Category Data Name Date Amount Data Memo	<Savings> <Vacation> <Hawaii> <2018-02-01> <\$200> N/A
3	Main Category Sub Category Data Name Date Amount Data Memo	<Sports> <Golf> <gloves> <2019-08-15> <\$15> N/A
4	Main Category Sub Category Data Name Date Amount Data Memo	<Food> <Chinese> <2019-05-15> <\$8.5> N/A
5	Main Category Sub Category Data Name Date Amount Data Memo	<Salary> <Ippudo> <2019-06-15> <\$400> N/A

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Firstly, navigate to add data page by clicking the + button on the bottom right of the main page	Add Data page should open		
2	Navigate to delete data page by clicking the "Delete Data" button on the top right	Delete Data page should open		
3	Choose the type of the new financial data by clicking either the Expenses / Income tab on the top	- Clicking the Expenses Tab should display input fields for deleting existing expenses financial data - Clicking the Income Tab should display input fields for deleting existing income financial data		
4	Choose Main Category from the dropdown lists located on the left-hand side of the page	Clicked variable in dropdown should be selected as main category and show in the user input field labeled as main category		
5	(Choose Sub Category from the dropdown lists located on the left-hand side of the page, only If necessary)	Clicked variable in dropdown should be selected as sub category and show in the user input field labeled as sub category		
6	(Type Data Name to delete if the user already knows which financial data he or she wishes to edit)	Credentials can be entered		
7	Click "Reveal List of Expenses / Income Data to Delete" Button	After a page reload, list of expenses / income that can be deleted shows up - If sub category input field is left blank and gets submitted, data list should show every data that is the selected main category's sub category as well Any lists should be clickable		
8	Click one of the lists that needs to be edited			
9	Click "Delete Your Expenses / Income Data" Button	One of the two messages should be shown: "Are you sure you want to delete ---" or "Error"		
10	Click "Delete"	Redirect to success page with information of deleted financial data		
11	Click "Ok"	Redirect to main page		

2.3. <Calendar View>

2.3.1. <Calendar View>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_008	Calendar View	High	Test the Calendar View Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	(Ideally, financial data entered by this user exists, for testing purposes)

Test Scenario

Verify on navigating to the MyCalendar page, the user gets to view the calendar of the current month

S#	Variable	Test Data	Notes
1	Day	<March 11>	
2	Day	<March 31>	
3	Day	<Blank Space on MyCalendar>	
4	Left Arrow clicked	** A Month Before	
5	Right Arrow Clicked	** A Month After	

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to MyCalendar Page through the side menu bar	Site should open		
2	Click on dates specified	Date buttons should be clickable - If clicked day has data entered in, then show all data lists available for this specific day - Else show message that there is no data added on this specific day		
3	Click Arrows on the top right or the top left go back and forth between the months	- If arrow button on the top left has been clicked, calendar should show a month before previous month - If arrow button on the top right has been clicked, calendar should show a month after previous month		

2.4. <Financial Category>
2.4.1. <Category List>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_009	Category List	High	Test the “Display the Category List” Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page

Test Scenario

Verify on clicking the “category” on the side menu bar, the user can view all available financial categories for him or her on the MoneyPlanner.

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to category page through the side menu bar	Category list page should open		
2	Click on buttons with category name on it	- Detailed category list information gets displayed on the right-hand side of the page		

2.4.2. <Add Category>

Test Case ID Module Name		Test Priority (Low/Medium/High)	Test Case Description		Test Case (Pass / Fail / Not Executed)
MP_010	Add Category	High	Test the “Add A New Category” Functionality in MoneyPlanner		Pass... and Fail

Preconditions:	
1	User is signed in
2	User is on the main page

Test Scenario

Verify on the fact that the category that will be created will not have the same category name as any of existing ones, the user can add a new user-specific financial category to the MoneyPlanner.

Notes

- All fields except for “Category Memo” are required fields
- Invalid inputs will be entering category names that already exist either as default category or user-specific category.

Variable		Test Data	
S#	Variable		
1	Type	<Expenses>	
	Main Category Name	<Sports>	
	Sub Category Name	N/A	
	Category Memo	N/A	
2	Type	<Expenses>	
	Main Category Name	<Food>	
	Sub Category Name	N/A	
	Category Memo	N/A	
3	Type	<Expenses>	
	Main Category Name		
	Sub Category Name		
	Category Memo		

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to add category page through the side menu bar	Category list page should open		
2	Choose the type of the new financial category by clicking type tab	Clicking the Expenses / Income Tab should display input fields for adding new expenses / income financial category accordingly		
3	Click the + button on the bottom left	Input fields for adding category shows up		
4-1	(If adding a new main category) Enter a new main category name that is not listed, and category memo if desired	Credentials can be entered		
4-2	(If adding a new sub category) Enter main category name that is not listed, as well as sub name and category memo	Credentials can be entered		
5	Click "Add Your Category"	One of the two messages should be shown: "Successfully Added" or "Error"		
6	Click "Ok"	Redirect to main page with the added financial data reflected on charts		

2.4.3. <Edit Category>

Test Case ID		Module Name	Test Priority (Low/Medium/High)		Test Case Description		Test Case (Pass / Fail / Not Executed)
MP_011		Edit Category	High		Test the “Edit Existing Category” Functionality in MoneyPlanner	Pass ... and Fail	

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	(Ideally, financial data entered by this user exists, for testing purposes)

Test Scenario

Verify on the fact that it is a user-specific category, not a default category, the user can edit the financial category in the MoneyPlanner,

Notes

- All fields except for “Category Memo” are required fields
- Invalid inputs will be entering category names that already exist either as default category or user-specific category.
- Valid inputs are handled well, when erroneous inputs are put in, after redirecting to the original page, it will not show category list anymore

Variable		Test Data
S#		
1	Type Main Category Name Sub Category Name Category Memo	<Expenses> <Sports> N/A N/A
2	Type Main Category Name Sub Category Name Category Memo	<Expenses> <Food> N/A N/A
3	Type Main Category Name Sub Category Name Category Memo	<Income> <Salary>

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Firstly, navigate to category list page by clicking on "category" on the side menu bar	Category list page should open		
2	Navigate to edit category page by clicking the "Edit Category" button on the top right	Edit category page should open		
3	Choose the type of the financial category to edit by clicking type tab	Clicking the Expenses / Income Tab should display input fields for editing expenses / income financial category accordingly		
4-1	(If Editing a main category) Choose main category from the dropdown list located on the left-hand side of the page	Clicked variable in dropdown should be selected as main category and show in the user input field labeled as main category to be edited		
4-2	(If Editing a sub category) Choose main category from dropdown list and then choose sub category from the other dropdown list	- Clicked variable in main category dropdown should be selected as main category and show in the user input field labeled as main category - Clicked variable in sub category dropdown should be selected as sub category and show in the user input field labeled as sub category		
5	Enter Category Memo if desired	Credentials can be entered		
6	Click "Edit Your Expenses / Income Category" Button	After a page reload, one of the two messages should be shown: "Successfully Updated" or "Error"	Not successful, the system throws SQL Exception	
7	Click "Ok"	Redirect to main page with the updated financial data reflected on charts		

2.4.4. <Delete Category>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_012	Delete Category	High	Test the “Delete Existing Category” Functionality in MoneyPlanner	Fail

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	(Ideally, financial data entered by this user exists, for testing purposes)

Test Scenario

Verify the fact that it is not any of the categories listed below in Notes section, the user can delete the category from the MoneyPlanner.

Notes

S#	Variable	Test Data
1	Type Main Category Name Sub Category Name Category Memo	<Expenses> <Sports> N/A N/A
2	Type Main Category Name Sub Category Name Category Memo	<Expenses> <Food> N/A N/A
3	Type Main Category Name Sub Category Name Category Memo	<Income> <Salary> N/A N/A

Category that CANNOT be deleted:

- Default Main / Sub category
- User-specific Sub category that has data in it
- User-specific main category that either has data in it or it is a parent category of sub category that has data in it
- ** This operation is not successful because of the constraint of database foreign keys

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Firstly, navigate to category list page by clicking on "category" on the side menu bar	Category list page should open		
2	Navigate to edit category page by clicking the "Delete Category" button on the top right	Delete category page should open		
3	Choose the type of the financial category to delete by clicking type tab	Clicking the Expenses / Income Tab should display input fields for deleting expenses / income financial category accordingly		
4-1	(If Deleting a main category) Choose main category from the dropdown list located on the left-hand side of the page	Clicked variable in dropdown should be selected as main category and show in the user input field labeled as main category to be deleted		
4-2	(If Deleting a sub category) Choose main category from dropdown list and then choose sub category from the other dropdown list	- Clicked variable in main category dropdown should be selected as main category and show in the user input field labeled as main category - Clicked variable in sub category dropdown should be selected as sub category and show in the user input field labeled as sub category		
5	Enter Category Memo if desired	Credentials can be entered		
6	Click "Delete Your Expenses / Income Category" Button	After a page reload, one of the two messages should be shown: "Successfully Updated" or "Error"	Not successful, the system throws SQL Exception	
7	Click "Ok"	Redirect to main page with the updated financial data reflected on charts		

2.5. <Initialize / Update Budget>

2.5.1. <Initialize/Update Budget>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_013	Initialize Budget	High	Test the “Initialize Budget” Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	(Ideally, financial data entered by this user exists, for testing purposes)

S#	Variable	Test Data	Notes
1	Updated Budget	<\$10>	
2	Updated Budget	<\$-10>	Input is invalid if amount is less than 0

Test Scenario

Verify on entering valid budget information, the user can initialize budget plan in the MoneyPlanner

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to add budget page by clicking the “Budget” button on the side menu bar	Budget home page should open		
2	Choose the type of the financial category by clicking type tab	By clicking one of the category buttons, it should show its current budgeted amount for the month		
3	Type the new budget amount for this category into input fields	Users should be able to type in values		
4	Hit the “SET” button	Should send response to success_add_budget.jsp or show error message and redirect		

2.6. <Initialize / Update Saving Goals Target>
2.6.1. <Initialize/Update Saving Goals Target>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_014	Initialize Savings Target	High	Test the “Initialize the Savings Target” Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page
3	(Ideally, financial data entered by this user exists, for testing purposes)

Test Scenario

Verify on entering valid budget information, the user can initialize saving goals target in the MoneyPlanner

S#	Variable	Test Data	Notes
1	Amount From To	<\$3000.0> <2019-02-15> <2019-05-15>	
2	Amount From To	<\$-3000> <2019-02-15> <2019-05-15>	Input is invalid if amount is less than 0

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to add saving goals target page by clicking the “savings target” button on the side menu bar	Saving goals target home page should open		
2	Enter the amount of saving that user is looking to save along with dates	Users should be able to put inputs into text fields		
3	Click the “Save Your Saving Goal”	Saves the results and data successfully gets inserted or forward response back with error message		

2.7. <Saving Tips>

2.7.1. <View Saving Tips>

Test Case ID	Module Name	Test Priority (Low/Medium/High)	Test Case Description	Test Case (Pass / Fail / Not Executed)
MP_015	View Tips	High	Test the “View Saving Tips” Functionality in MoneyPlanner	Pass

S#	Preconditions:
1	User is signed in
2	User is on the main page

Test Scenario

Verify on clicking the “Saving Tips” on the side menu bar, the user gets access to all saving tips within the MoneyPlanner

Step#	Test Steps	Expected Results	Actual Results (If different from Expected Results)	Status (Pass / Fail / Not Executed)
1	Navigate to view saving tips page by clicking “Saving Tips” on the side menu bar on the main page	- Site should open - Display list of all saving tips titles		
2	Click saving tips title	Content of each saving tip title expands to reveal saving tips		

7. Challenges Overcome

<MoneyPlanner>

Milleina Ohara

7. Challenges Overcome

This is a list of challenges overcome:

- **Server and Database connection setup**

A lot of research was needed to be done as to how to connect my Java application to web server and database server. I have determined to use Apache Tomcat server, a servlet container server that can serve dynamic contents like Java programs, and MySQL database server due to their popularity. With all initial attempts to connect these servers to the application, I was getting all types of HTTP request errors such as 404 and 500. Reading error logs to figure out what the actual root error was and looking up solutions online and make attempts to fix the issue took an enormous amount of time.

Steps that solved the issue were to make sure that there were jar files like `servlet-api.jar` and `my-sql-connector.jar` imported in the proper directory of the application and configuring the build path to it. In addition to it, I found the correct server mapping was crucial. Traditionally, servlet mapping can be done by configuring servlets within a `web.xml` file. I have ended up using servlet annotation that is a fairly new technology and was a definitely cleaner solution and easier to understand.

- **Database structure and optimization**

In order to store money transactions information into database, several database tables were required for transaction information itself and category information of which each transaction belongs to. Initially, two separate database tables were created to handle expenses and income category as well as two more database tables to expenses and income transactions with unique categoryIDs assigned to each transaction data. With this original structure of the database, database tables that handle expenses transactions would have been excessively more overloaded compared to the income database. After having discussion with Dr. Hayes, I have optimized the structure by combining expenses and income transactions database tables together. The way it has been done was to have the two category databases integrated into one as well and add type variable to this table in order to indicate whether the specific category handles expenses or income transactions. And from there, because each category has unique IDs to it, any transactions can point to those categoryIDs to find out what type of transaction it is.

Since I had separate category database tables for main category and sub category, I optimized them as well by integrating those two tables into one by using the similar solution as above; more specifically, integer variable `mainID` was added to the integrated table to indicate whether the category was a parent or a child category. If `mainID` had a value 0 then it indicates that the category is a main category and each sub category gets assigned a `mainID` that corresponds to the parent category's categoryID.

- Getting used to MySQL query statements

Learning to use database queries was perhaps one of the most fun parts of the project. Utilizing statement interfaces, resultset, and different queries like SELECT, INSERT, and UPDATE, most of data retrieval process was producing expected dynamic results after a few times of experiments and error checking. One error that took me long to find its solution was when retrieving the data list, it used to always display the content from the second element of it. The core problem of this was that every time I checked whether the certain result set was empty or not, cursor gets moved to after the first element because the only first element gets checked in order to find out if the result set is empty or not. So the solution was to use ResultSet's `beforeFirst()` method to set cursor to before the first element in the set so that data retrieval procedure can be done properly right after.

- Determining whether to use PHP

PHP being the server-side scripting language that can pull out database contents and JavaScript being the client-side language, it would have been ideal to use both of web programming languages to achieve dynamic results on the browser. Especially since I wanted to use Chart.js API for my interactive and dynamic charts based on data received by the server, PHP was a classic way of achieving the goal. The problem I had with this solution was that I was unable to get PHP working the way it was expected with Java. I was able to put both Java files and PHP files together in the same project and get them to work individually. After installing JavaBridge jar file and configuring AJP protocols which presumably allow communication between Java and PHP, they still seemed to be not communicating. As much as the desire to get this solution to work, as time was running out, I have ended up deciding not to use PHP and instead, I had Java Servlets execute queries and retrieve contents of database, forward the retrieved data as HTTP responses to Java Server Pages (JSP), and JavaScript retrieves it from JSP through JSP scriptlet tags and JSP expression tags.

- Getting JavaScript to display dynamic elements without PHP

Because I have decided not to use PHP to get database contents, I had to take the longer way to display dynamic results using Java Servlets, JSP and JavaScript. MoneyPlanner application required elements such as buttons and dropdown-lists with dynamic IDs and values assigned to them depending on the database contents such as a list of categories that might be pre-set or added by user. Forwarding the list of data from servlets and to JSP was done easily by HTTP requests; however, passing the array of contents from JSP to JavaScript was tricky and had to be done manually in my solution. Which is to concatenate all data in a list into a single string, pass it to JavaScript and break it down into an array and output from there to create elements. This method gets the job done but I would assume there would have probably been a lot easier and smart way of achieving the same result.

8. Future Enhancements

<MoneyPlanner>

Milleina Ohara

8. Future Enhancements

This is a list of future enhancements to be made:

- Make it a cross platform application

One of my main goals of this project was to make it a cross platform application that can be used by desktops, laptops, and smartphones. I have only got to develop laptop version of the application and I assume that making the interface appearance convertible to the mobile version would be tricky but definitely one of the major things I would like to work on in the future.

- Email authentication / password reset

As of right now, the application does not include features to validate email that was used in registration process and it can be any random string. In the future, I would have the validation sorted out and also in case of a user forgetting their password to sign in to the application, it will send the notification mail to let the user reset the password if desired by the user.

- Send notifications/reminders/warnings if you're approaching budget limits

Because there is no real email validation made available to this application, sending reminders of the user approaching the budget limit is not possible. Also, after developing the mobile version of the application, I could show notifications within the application instead when the user is online.

- Budget prediction

By analyzing the user's spending habits, the application will provide the user more detailed insights of where they could improve. I could also include features like top three ranked spending categories.

- Advanced features in Calendar view

I think it will be a cool feature to be able to enter recurring future transactions and bills such as paycheck and utility bills by category and display them in calendar view so the user can get at-a-glance view of upcoming expenses. The application should also show the projected balance so the user knows exactly how much money can be spent

- Category

By marking the pre-set category and user-specific category differently somehow, the user will not have to go through the step to make attempts to edit or delete the default category only to find out the process is impossible.

- Showing recent transactions

Feature to show the top three most recently added information is desirable.

- Saving tips feature

On top of saving tips that are already available in the application, it might be good to have a feature where the user can also add his or her own saving tips. Also, a feature where you can “like” saving tips by clicking a button next to each of the tip and review them in a “your favorite saving tips” would be cool. I would also include a more advanced functionality as well where the application suggests the user specific saving tips based on his or her spending habits by sending out the notification.

- Several different patterns of color scheme

Choosing the color scheme of the user interface was definitely time consuming. I have tried multiple different color patterns from darker to brighter and from minimal style to more complex coloring scheme. It would be very nice to have various color schemes that each user can choose from to try out which pattern he or she likes the most.

- More research on HCI (which input fields to use, specifically)

Although one of the main focuses of this project was on the appearance and effectiveness of the user interface, I feel like I have not done enough experiments with different input styles to which one will give the best possible result. Also, I think there is a room for improvements in layout of the application as well.

- Export data to different file formats

This functionality was included in the initial project proposal as more of an advanced feature before planning and setting up overall schedule and milestones.

9. Final Defense Presentation Slides

<MoneyPlanner>

Milleina Ohara

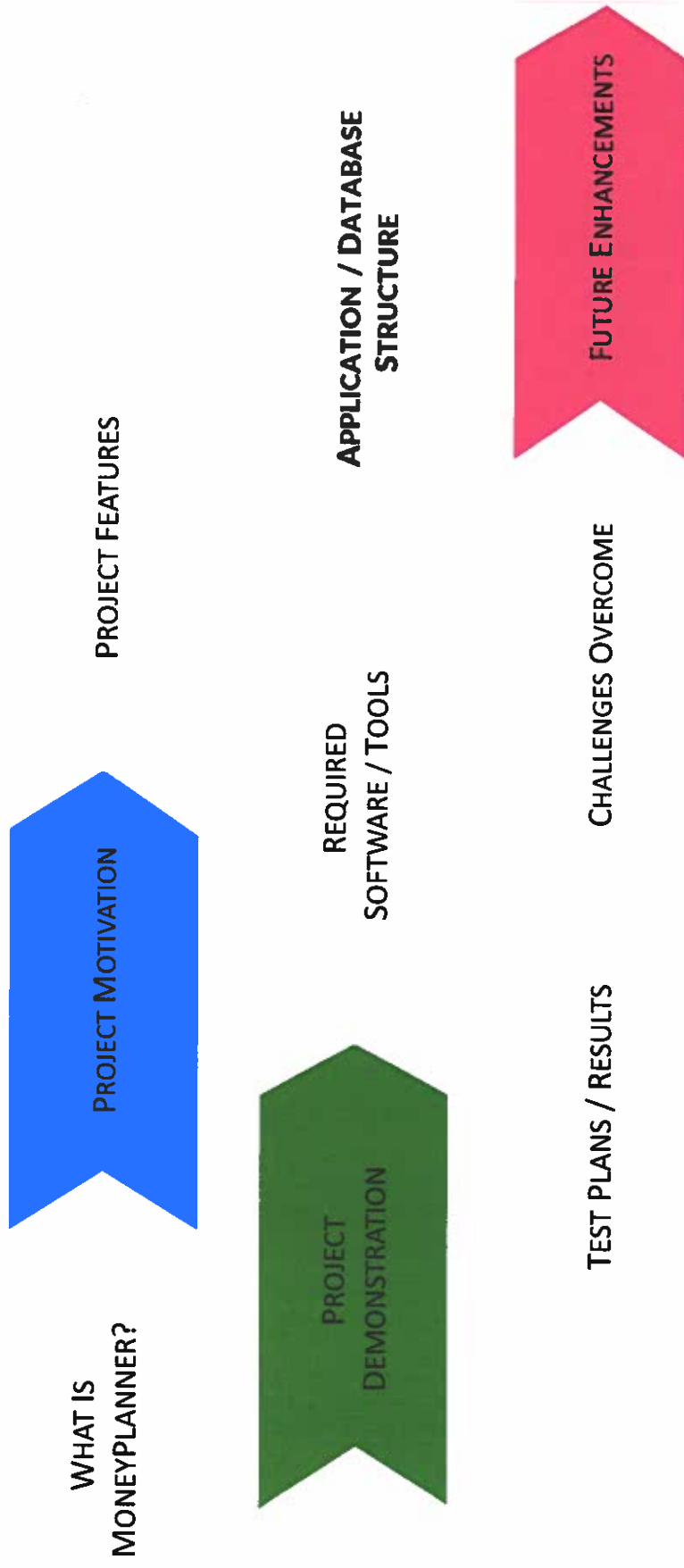
MONEYPLANNER

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MILLEINA OHARA

Presentation Flow

1 of 11



WHAT IS MONEYPLANNER?

2 of 11

Web-Based Application

To Help You **Organize Your Finances**

1. Track Your Expenses
2. Create A Budget Plan
3. Saving Goals

WHY IMPORTANT?

Key Points: **Intuitive & Interactive UI**
Easy to Use and Learn

Better Demonstrate How You're Using Your Money

PROJECT MOTIVATION

REASON 4

Experience in Building
A Web Application
Development

REASON 3

Desire to Get More
Familiar with Java and
Database

REASON 2

Interests in HCI
and UI Design

REASON 1

Personally I Needed
A Finance App

PROJECT FEATURES

TRACKING EXPENSES

Total Expenses Sorted By Category
Along With Total Balance and Total
Income

CREATING A BUDGET

Create Or Update Your Monthly Budget
Plan For Each Category!

FEATURES

02

01

03

CUSTOMIZABLE
CATEGORY

No More Adjusting Your Life Style To
Pre-Set Patterns Of Categories... Create
Your Own!

04

SAVING GOALS

Set Your Saving Goals For Various
Purposes and See Your Savings Grow!

05

SAVING TIPS

MoneyPlanner Provides Some Useful
Saving Tips to Help You Stay On Track

PROJECT DEMONSTRATION

HOW IT IS BUILT

MoneyPlanner

REQUIRED SOFTWARE / TOOLS

SERVER

DATABASE

PROGRAMMING
LANGUAGES

TOOLS

Apache
Tomcat Server

MySQL Database

[SERVER-SIDE]

Java
SQL

Eclipse IDE for JAVA
EE Developers

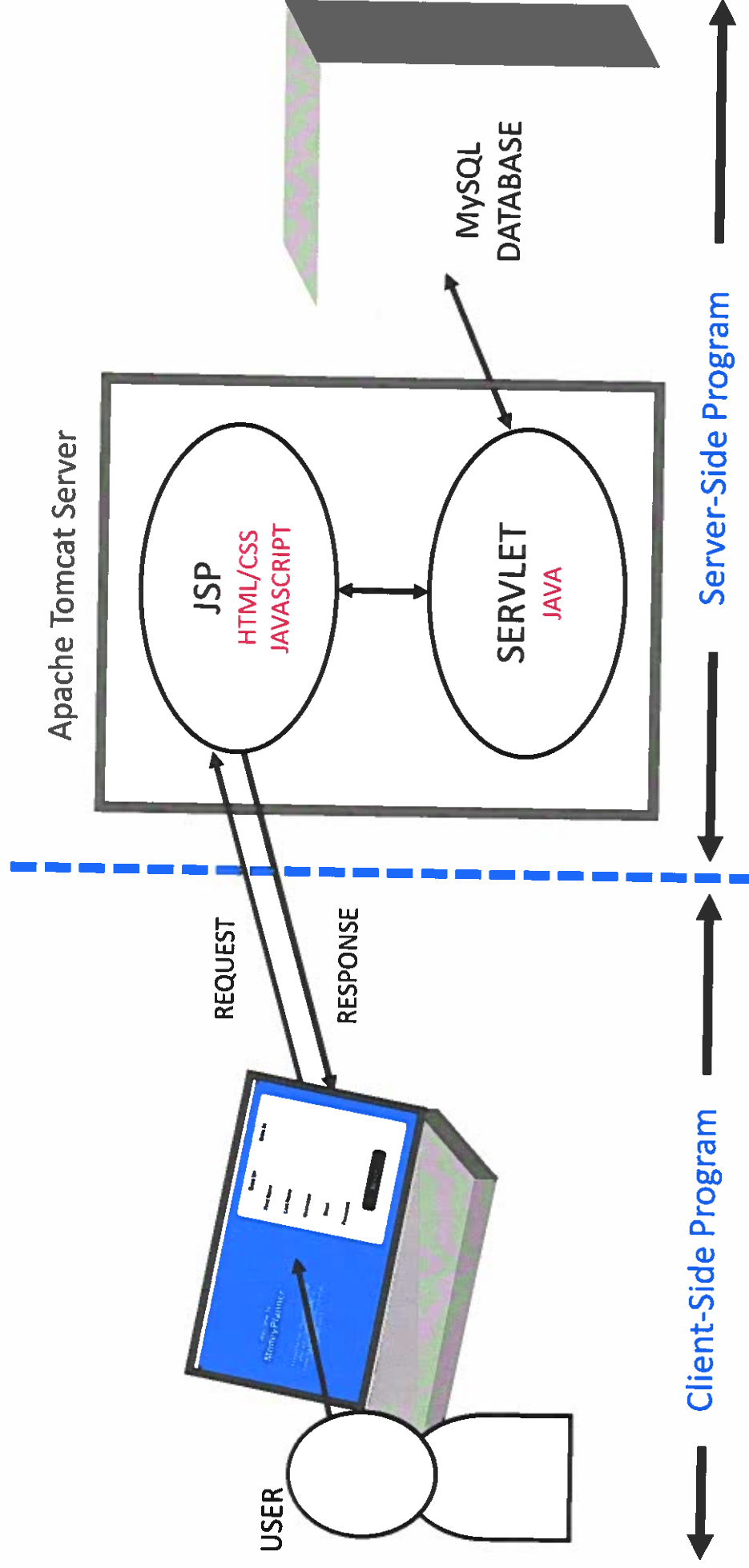
[CLIENT-SIDE]

HTML
CSS
JavaScript

MySQL Connector/J

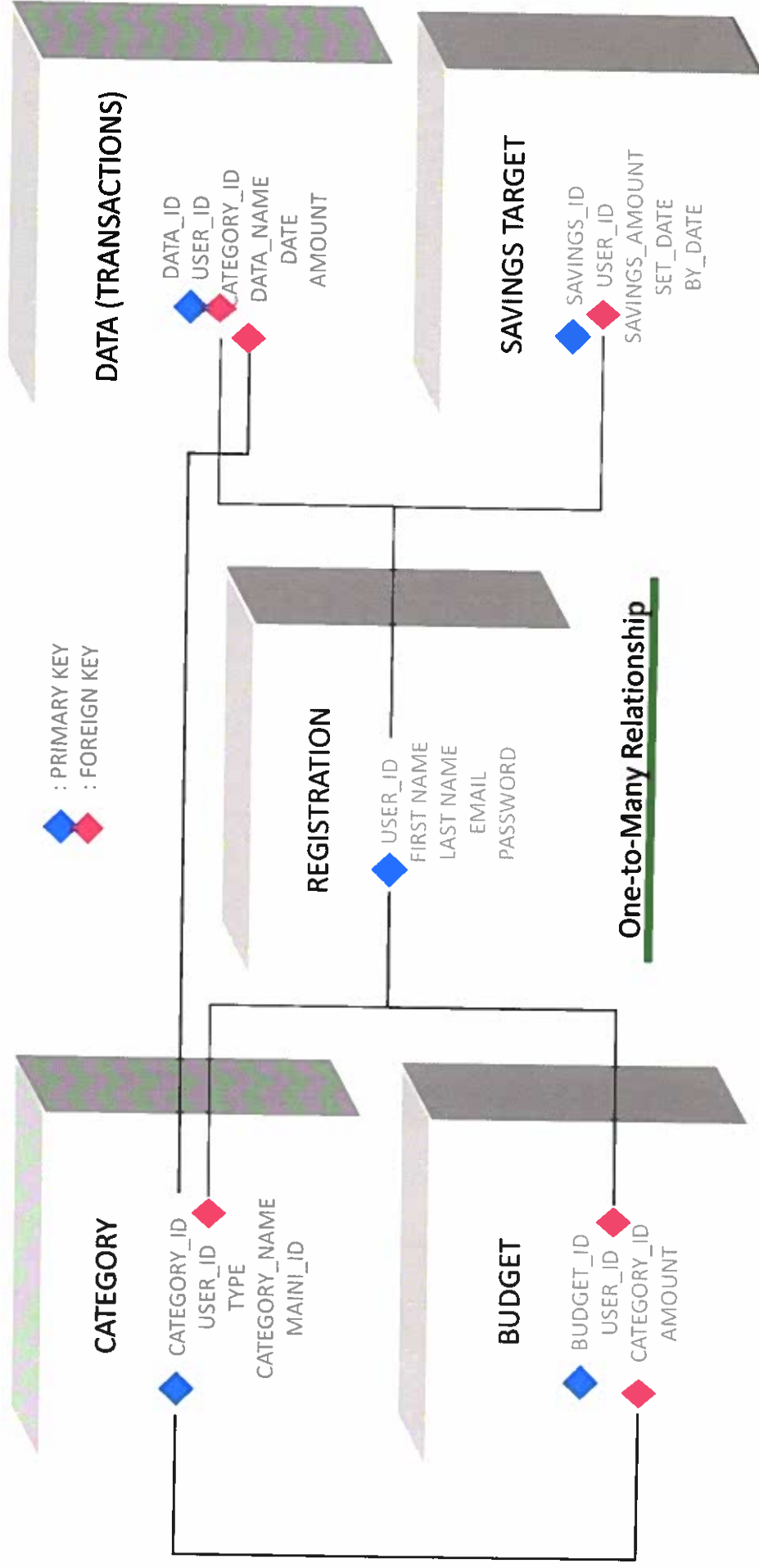
Application STRUCTURE DIAGRAM

7 of 11



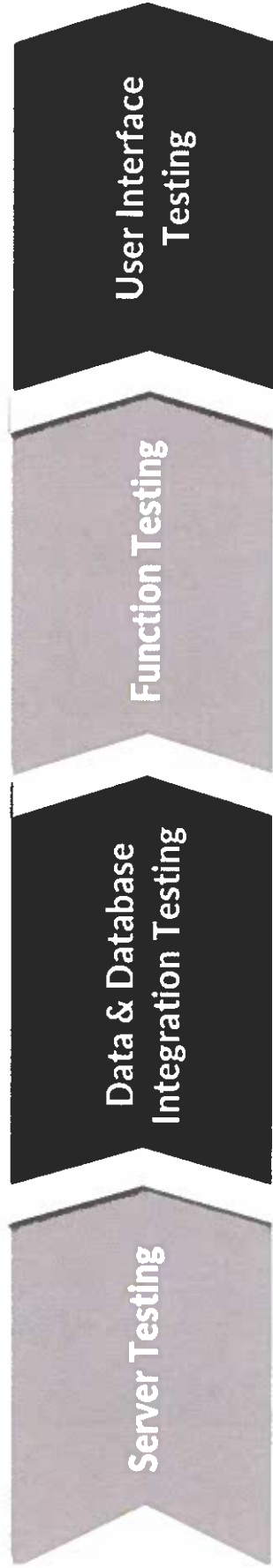
DATABASE STRUCTURE

Many-to-Many Relationship



TEST PLAN AND RESULTS

Test Cases Pass / Fail Rate from 4 different participants



100%



Server Connection
Success Rate

100%



Database Operation Success
Rate

87%



Calculation / Operation
Success Rate

Questionnaire
Results

CHALLENGES OVERCOME

WHAT I NEEDED TO OVERCOME

WEB SERVERS CONNECTION SETUP

Apache HTTP Server
Apache Tomcat Integration
Java Server Pages (JSP)
Proper Servlet Mapping
Using Servlet Annotation @

DATABASE DESIGN AND OPTIMIZATION

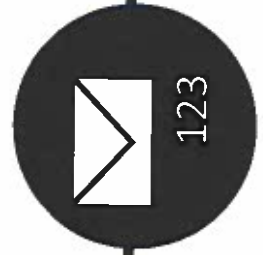
One-to-Many Relationship
Many-to-Many Relationship

CREATING DYNAMIC DOM ELEMENTS

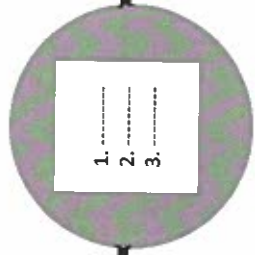
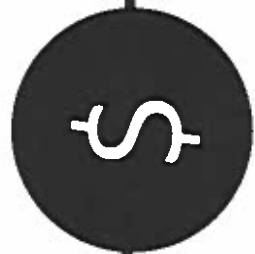
With JavaScript
Buttons
Tables
Dropdown Lists

FUTURE ENHANCEMENTS

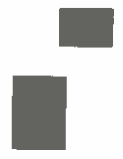
Email & Password
Authentication



Budget Prediction



Cross-Platform
Desktop + Smartphones



Color Scheme
Patterns



Recent
Transactions



ANY QUESTIONS?